GESP C++ 五级 · 小题学习讲义

理解 · 手推 · 验证 · 迁移

从“认得算法”走到“说得清为什么”

依据九套五级试卷的 225 道小题与所提供考纲整理。沿用四级讲义的讲法：从一个具体例子出发，讲清变量和过程，再用新例子检验。

9	225	28	25
套试卷	道真题索引	个主题课	道自编验收题

给孩子

先跟着例子弄懂发生了什么，再读方法整理，最后自己做 T 任务。一个专题可能跨几页，不必一次学完；能解释一步为什么成立，比赶到下一页更重要。

给家长

这是一份五级知识学习稿，不是孩子的错题诊断。尚无五级作答记录，所有掌握状态均待验证。先让孩子尝试，家长参考放在末尾，不替她写反思。

材料范围：2406、2409、2412、2503、2506、2509、2512、2603、2606 五级卷

考纲依据：所提供《考纲.pdf》PDF 第 22–24 页

修订日期：2026 年 10 月 9 日 · 版本 2.1 · 详细讲解版（专题编号）

使用形式：可打印讲义 + 可折叠家长附录的网页

怎么学，以及学到什么程度

考纲五级分为八个知识块。复杂度贯穿每一种算法；本讲义先给工具，再按理解依赖安排主题。

考纲知识块	对应主题	掌握证据
复杂度估算	01、03、04、22	数实际操作和最大同时调用层数
递归算法	02—04	说清子任务、终点、返回过程和重复计算
初等数论	05—10	解释整除、同余、GCD、分解与筛法
链表	11—14	画箭头完成创建、增删、遍历和反转
二分算法	15—17	写清候选范围、单调性、收缩和返回值
分治算法	18—22	区分划分、递归、合并与各项复杂度
贪心算法	23—24	提出选择规则，并解释为什么不吃亏
C++ 高精度	25—28	按位解释加减乘除及进位、借位、余数

> **每次约 20—30 分钟，走完一段理解过程**
> 先读具体例子，能复述过程后再读方法整理；随后独立做 T 任务，最后选 1—2 道真题验证。一个专题可以分几次，不按页数追进度。学习时间纳入已有安排，不额外加量。

建议先后顺序

01—04 → 05—10 → 11—14 → 15—17 → 18—22 → 23—24 → 25—28。高精度可与“数楼梯”等正在学习的题同步，链表可接已有《链表探险指南》。

三档记录，不凭“看懂了”判会

标记	实际表现	下一步
待学	还说不清过程	先手推，不急着选项
练习中	有提示能做，换例子会混	换一个最小反例再解释
能迁移	无提示做对，能说明理由与边界	隔 2—3 天复测一题

“反转”“同余”“二分答案”“递归优化”等即使在旧卷中题量少，也因考纲明确要求而保留。真题频次仅描述给定材料，不是未来押题概率。

从哪里读起 · 八个知识块

同一专题，共用同一个编号与页眉名称。例如12.1、12.2都属于“专题12 | 链表插入”；小数点后是该专题的阅读顺序。页眉另标“第几页 / 共几页”，页脚是全书页码。

知识块	专题	PDF页码	阅读线索
复杂度	01—01	4—5	先数动作，再认识O与log
递归与优化	02—04	6—12	任务分工→调用栈→重复计算与复用
初等数论	05—10	13—25	因子配对→GCD→分解→同余→两种筛法
链表	11—14	26—34	位置盒子→改箭头→删除→循环与反转
二分算法	15—17	35—42	候选区间→第一个满足→二分答案
分治与排序	18—22	43—51	合并信息→归并→划分→分别数工作量
贪心算法	23—24	52—56	反例→交换理由→四种典型模型
高精度运算	25—28	57—65	位权→进位→借位→部分积→长除法

> **怎样接着四级那套讲义读**
> 先读链表11—14：其中的“指针位置盒子”直接接上四级。若希望按全书顺序学习，从复杂度与递归起步；遇到O或log不熟，可随时回专题01。

一节课里，哪些地方是给你自己想的?

“例子讲解”是教练带着讲；“整理与练习”把方法收拢，页末T任务由你独立完成。同一专题可以分几次读，不按页数赶进度；能自己画过程和说依据，才是学习证据。

配套材料	PDF页码
自编迁移验收	66—68
孩子自己写的复盘卡	69
九卷225题的专题索引	70—74
家长核对与材料校勘	75—84（建议单独保管）

打印给孩子时可选第1—74页；家长附录从第75页开始。网页默认收起家长附录，顶部菜单可跳转各页。

01.1 先把 8 张卡片摆出来，再谈“快不快”

假设桌上有 8 张卡片，你想找出最大的数字。先把第一张作为目前最大数，再从第二张起逐个比较。这里的“工作”，是一次比较，不是代码写了几行。

如果卡片增加到 16 张，还是用这个办法，就要多看大约一倍的卡片。增加到 100 张，也只是按同样方式继续看下去。我们关心的是：**数据变多时，工作量怎样跟着变多。**

先把每一轮真正做的次数写出来

办法	8 张卡片时	16 张卡片时	变化
从第二张起与目前最大数比较	7次比较	15次比较	数据翻倍，工作约翻倍
每张都与每张比较一次	8×8=64	16×16=256	数据翻倍，工作变4倍
每次把剩余范围缩成一半	8→4→2→1	16→8→4→2→1	数据翻倍，只多缩一次

第三行先数的是“缩小次数”。实际查找是否还要检查最后那一张，要看代码；但“多一倍数据，只多一轮”的增长特点不变。

现在才给这些增长方式起名字

第一种记作 O(n)，第二种记作 $O(n^2)$。第三种的轮数接近“2 乘多少次才到 n”，这个次数叫以 2 为底的对数，记作 $\log_2 n$。因为 $2\times2\times2=8$，所以 $\log_2 8=3$。

> **O 不是秒数，也不是精确次数**
> O(n) 表示工作量至多按 n 的某个常数倍增长。n−1、2n、3n+5 在本阶段都按线性量级看。比较两个 O(n) 程序时，仍可能一个快一个慢；机器、常数与实现细节并没有消失。

所以不要先问“这题应该背哪个 O”。先找一个需要反复做的动作，列出数据变多后它做几次，再把这个趋势写成符号。

> **接着往下读**
> 带着“数实际动作”读后面的复杂度表；T01 再由你自己数翻倍循环。

01.2　复杂度：数工作量，不数代码行数

这一节要弄懂什么
题目问“多快、多占空间”时，先确定 n 指什么，再看它变大后要多做多少事。

把循环翻译成工作量

动作	次数或总量	常用量级
从 1 数到 n	n 次	O(n)
1、2、4、8……直到超过 n	每次翻倍；n=16 时只需几步	O(log n)
外层 n 次，内层每次 n 次	n × n	$O(n^2)$
第 i 轮做 i 次	1+2+…+n	$O(n^2)$
每层总工作 n，约 $\log_2 n$ 层	n × $\log_2 n$	O(n log n)

算式规则
先后执行的工作量相加，保留增长最快的一项；真正嵌套且独立的次数相乘。两层循环不一定是 $O(n^2)$，如内层从当前指针继续向前，总移动也可能只有 n 次。

时间与空间分开问

时间看累计做多少事；辅助空间看**同一时刻**最多额外保留多少东西。输入数组本身通常不计入辅助空间；递归栈、临时数组要计入。

```
for (int i = 1; i <= n; i *= 2) {
    // 一次固定大小的工作
}
```

本讲义的运算次数估计默认整数运算 O(1)、数值不溢出；高精度专题另按位数计。若原题问平均、最好、最坏情况，必须分别回答。

独立任务 T01
令 n=16，列出上面循环中 i 的全部取值。再判断“外层 i 翻倍，内层每轮做 n 次”的时间量级，并说明为什么。

回到原卷：2606 判8；2603 判4、判7。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

02.1 递归里的“另一个自己”，究竟拿走了什么任务?

我们先只做一个很小的任务：计算 1+2+3+4。你当然可以直接加；现在换一个分工方式，目的是看懂递归怎样工作。

约定 S(n) 负责“算出从 1 加到 n 的和”。那么 S(4) 可以留下最后的 4，把前面的 1+2+3 交给 S(3)。只要 S(3) 把自己的任务完成，S(4) 再加上 4 就行。

S(4) 的整份任务：1+2+3+4

交出去的部分：1+2+3　　自己留下的部分：4

这不是随便把答案叫作 S(3)。我们已经给 S(3) 规定了明确职责，它只需要解决一个**同样的、但更小的**问题。

先把任务交下去，上一层先等着

这一层	留下什么	等待谁
S(4)	最后还要加4	S(3) 的结果
S(3)	最后还要加3	S(2) 的结果
S(2)	最后还要加2	S(1) 的结果
S(1)	直接知道答案是1	不再交出任务

到了 S(1)，不需要再交给别人。它把 1 交回 S(2)，S(2) 算出 2+1=3；再交给 S(3)，得到 3+3=6；最后 S(4) 得到 4+6=10。

> **最容易漏掉的动作**
> 调用下一层时，这一层还没完成。它保留着“回来后还要加几”这张便条。下一层返回后，要回到原来暂停的位置继续，不能当成整份程序重新开始。

“从大任务交到小任务”是去程，“把小任务答案逐层带回来”是回程。递归表达式写得短，是因为它把这两段过程藏在调用和返回里了。

> **接着往下读**
> 接着把这个分工写成递归说明卡；T02 会换成阶乘，检验你是否真的懂了任务分工。

02.2　递归：把同一种小任务交给下一层

这一节要弄懂什么
先理解一次调用承诺完成什么，再看它怎样交出更小任务。递归不是“代码长得像函数套函数”。

三个问题组成一张递归说明卡

问题	求 1+2+…+n 的例子
这次调用负责什么?	S(n) 返回前 n 个正整数的和
最小任务直接怎么办?	S(1)=1
怎样交出更小任务?	S(n)=n+S(n−1)，下一层负责前 n−1 个数

分清“去”和“回”
S(4) 等 S(3)，S(3) 等 S(2)，S(2) 等 S(1)。到最小任务后，把结果逐层带回来。每一层的 n 是这一层自己的参数。

```
S(4) = 4 + S(3)
            S(3) = 3 + S(2)
                        S(2) = 2 + S(1)
                                    S(1) = 1
```

终止条件写了，也必须真的走得到

若只在 n==0 时返回，却总调用 f(n+1)，从正数出发就越走越远。检查“终点”与“每次朝终点推进”是两件事。

`sin(sin(x))` 是嵌套调用；仅凭外面又写了一次函数名，不能说明 sin 的函数体在调用自身。递归可以直接调用自己，也可以经其他函数间接回到自己。

独立任务 T02
用同样的方法定义“n 的阶乘”任务：说明返回什么、最小情况、下一层负责什么，再画出 n=4 的去程和回程。暂时不写完整函数。

回到原卷：2409 选3、选15、判10；2512 选13、判9。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

03.1 一层一个便条：为什么返回后还能记得自己的 n?

仍看求和，但把一行想象成三个动作。下面只是函数内部的局部片段，假设 n>1：

```
int small = S(n - 1);  // 先等小任务完成
int result = n + small;
return result;
```

S(4) 调用 S(3) 时，不是把同一个 n 从 4 改成 3。新的调用有自己的参数 n；上一层的 n=4 仍留着。否则回来时，它就不知道还应加几。

便条①：n=4，等 S(3) 回来，接着算 4+small

便条②：n=3，等 S(2) 回来，接着算 3+small

便条③：n=2，等 S(1) 回来，接着算 2+small

便条④：n=1，直接交回 1

最后放上去的便条，先处理完

n=1 的那层先返回，n=2 才能继续，随后是 n=3、n=4。这种“后放上去，先取下来”的管理方式，就是递归调用栈。先把便条过程理解清楚，再记“栈”这个词。

如果 cout 写在调用前，它会在去程执行；写在调用后，它要等下一层完成后才执行。于是同一个 n 可能在两个不同时刻输出。

动作	别和什么混在一起
cout << n	在屏幕上留下一个数字，不等于把它交给上层
return result	把结果交给调用者；不会自动打印
函数被调用一次	新建一层任务；这层可能打印0次、1次或多次

看图数空间

最深处有4张尚未全部返回的便条，就要同时容纳4层。若之后还有另一条分支，不要把已经返回并撤掉的旧便条再次算入“同时占用”。

接着往下读

下一页用 walk 的前后两次输出练习；请分别追踪屏幕、返回位置和便条层数。

03.2　递归追踪：输出、返回值、调用次数是三件事

这一节要弄懂什么
读代码时先圈出题目到底在问什么，避免把树的深度当调用总数，把屏幕输出当返回值。

```
void walk(int n) {
    if (n == 0) return;
    cout << n << " ";    // 下去前做
    walk(n - 1);
    cout << n << " ";    // 回来后做
}
```

walk(2) 的时刻	发生的事
进入 n=2	先输出 2，等待 walk(1)
进入 n=1	先输出 1，等待 walk(0)
进入 n=0	直接返回，不输出
回到 n=1，再回 n=2	分别执行各自后面的 cout

数时间与数栈的方法
在调用树上，每出现一个函数调用就画一个节点；时间看整棵树的节点和节点内工作。空间看最长的未返回路径，不把所有分支同时相加。

同样是“递归”，成本可能很不同

调用方式（每次其余工作 O(1)）	时间	递归栈
只调用一次 f(n−1)	O(n)	O(n)
只调用一次 f(n/2)	O(log n)	O(log n)
调用两次 f(n−1)	$O(2^n)$	O(n)

三柱汉诺塔就是“搬走上面 n−1 个 → 移动最大盘 → 搬回 n−1 个”。移动次数满足 M(n)=2M(n−1)+1, M(1)=1，因此为 2^n-1；栈深只随 n 线性增长。

运算符左右两边都调用会输出的函数时，不应凭书写顺序断定 C++ 的求值顺序。本页用分开的语句明确先后；原卷相关问题见家长附录。

独立任务 T03
手推 walk(3) 的完整输出，再统计含 n=0 在内一共调用几次、同时最多有几层。请把三种答案分别标名。

回到原卷：2406 选14–15；2503 选7；2606 判8。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

04.1 为什么同一个 F(3) 会被算两遍?

斐波那契数列规定 F(0)=0、F(1)=1，后面每项是前两项之和。这个关系本身没有问题，慢的地方出在我们怎么执行它。

假如直接把每次 F(n) 都展开成 F(n−1)+F(n−2)，F(5) 的工作单会这样分开：

```
F(5)
├─ F(4)
│   ├─ F(3)
│   └─ F(2)
└─ F(3)   ← 这个参数前面已经出现过
```

两份 F(3) 的答案明明相同，但普通函数调用不会因为名字和参数相同，就自动记得上一次结果。每份都继续拆出自己的 F(2)、F(1)，于是重复越来越多。

给已经做完的任务留一个登记格

格子	空着时	填过后
memo[3]	F(3) 尚未算好	保存 F(3) 的答案
memo[4]	F(4) 尚未算好	保存 F(4) 的答案

第一次需要 F(3)，照常把它算完，然后填入 memo[3]。以后又遇到同一个任务，先查格子，已经填过就直接取走结果。这个办法叫记忆化。

> **为什么不能只说“加个数组就快了”**
> 数组存的是“哪个状态已经求完、答案是什么”。必须先查，再决定是否计算；算完后还要登记。只有数组声明，没有这套使用过程，不会减少重复。

对于这份数列，一共只有 0…n 这些不同参数。每个新状态的加法只算一次，因此固定宽度运算下工作量变成 O(n)。但递归仍可能一路走到最小参数，调用栈并没有因为记忆化就自动消失。

> **接着往下读**
> 接着看快速幂：它也有重复子任务，但只需要一个局部变量就能复用。

04.2 快速幂里的 half：同一份结果用两次

要算 3^8，可以从1开始连续乘8次3。现在观察：$3^8=(3^4)\times(3^4)$，而 $3^4=(3^2)\times(3^2)$。同样一半的结果可以重复使用。

如果要算 3^9，先拿两个 3^4 相乘，只覆盖了 8 个 3，还差一个 3。因此奇数指数要再乘一次底数。

先跟着一条路线算完

当前任务	交出去的一次小任务	回来后怎么做
3^8	求3^4	把结果乘自己
3^4	求3^2	把结果乘自己
3^2	求3^1	把结果乘自己
3^1	可以直接得到3	作为最小情况返回

回程是 3 → 9 → 81 → 6561。每一层只等一份小答案，把它保存在 half 里，再使用两次。

```
// 甲：求一次，结果留住
long long half = power(x, n / 2);
// 后面使用 half * half

// 乙：同一个小任务重新调用两次
// power(x, n / 2) * power(x, n / 2)
```

甲只沿一条减半链走。乙每层都分出两条：1 个任务变 2 个，再变 4 个。**每个任务虽然变小了，总任务数却在增加。**只看到 n/2 就判断 O(log n)，会漏掉这个区别。

> **两种“省”要分别检查**
> 复用结果减少调用总数，改善时间；但两种写法最深都沿着 8→4→2→1，因此栈深仍同为 O(log n)。这里假设中间数值不溢出。

本例指数为正整数，以n=1为最小情况。若允许n=0，还应单独返回1（底数非零时$x^0=1$），不能漏掉指数0这个入口。

> **接着往下读**
> 下一页把记忆化、递推和快速幂并排比较，再完成 T04 的两棵调用树。

04.3　递归优化：相同的事别重新算

> **这一节要弄懂什么**
> 看懂记忆化与快速幂的共同点：已经算出的结果能否复用。函数名字叫“快速”不代表真的快。

斐波那契：重复的是同一个状态

F(5) 需要 F(4) 和 F(3)，而 F(4) 又需要 F(3)。朴素递归会重新展开同一个任务。**记忆化**用表保存“这个参数的答案是否已经算过”。

实现	时间（按固定宽度运算）	辅助空间
朴素 F(n−1)+F(n−2)	指数级，常用上界 $O(2^n)$	O(n) 栈
记忆化：每个状态只求一次	O(n)	O(n) 表与栈
从小到大递推，只保留前两项	O(n)	O(1)

快速幂：先算一半，再复用

```
long long half = power(x, n / 2);
// n 为偶数：使用 half * half
// n 为奇数：还要多乘一个 x
```

> **同一个子任务写两次，不会自动替你缓存**
> 若写成 `power(x,n/2) * power(x,n/2)`，算法分析要计两次调用。一个减半子调用是 O(log n)；两个相同减半子调用的总节点数是 O(n)。

递归可以改成非递归，复杂过程可用显式栈模拟，简单过程可直接循环。**不保证所有递归都会被编译器自动优化**；尾调用优化也不是 C++ 必须提供的保证。

> **独立任务 T04**
> 从 n=8 出发，画“每次算两遍 n/2”和“只算一遍并复用”两棵调用树，基例 n=1。比较调用总数，并说明两者栈深为什么同阶。

回到原卷：2409 选8；2509 判3、判9；2603 判10；2606 选7。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

05.1 因子成对站：为什么过了平方根就不用再试？

把36个小方块摆成长方形，可以摆成1×36、2×18、3×12、4×9、6×6。每一种摆法都给出一对能整除36的正整数。

一对因子	较小的一个	较大的一个
1×36	1	36
2×18	2	18
3×12	3	12
4×9	4	9
6×6	6	6

接着试9×4、12×3，只是把前面的一对交换左右。到了6×6这个交会处，继续向后就不会得到一种全新的配对。

任意一个合数，也有这样的较小因子

若n=u×v，而u、v都大于√n，它们相乘就会大于n，不可能仍等于n。所以至少有一边≤√n。要证明n没有非平凡因子，只要检查2到√n这一段。

例如判断31，√31在5和6之间，只需排除2、3、4、5；其中4是2的倍数，若只试素数，则排除2、3、5就够。它们都不能整除31，后面也不可能藏着一对全都大于√31的因子。

> **等号是被平方数逼出来的**
> 若只检查d<√n，遇到25就漏掉5×5：2、3、4都除不尽，不代表25是素数。正确范围包含等号；代码常写d*d≤n，数大时可改用d≤n/d避免乘法溢出。

还有一个必须先处理的特例：1只有约数1，不符合“恰有两个正约数”的素数定义。不能因为试除循环一轮也没做，就把1当成素数。

同样的配对还能帮助数约数，但平方根处两边相同，只能记一次。对36，6是一个约数，不能因为写成6×6就登记两个不同的6。

> **接着往下读**
> 下一页再用定义区分素数、合数、互质和奇偶；T05用新数字检验试除范围。

05.2　数论起点：约数、倍数、素数与奇偶

这一节要弄懂什么
把名词变成可以检查的定义，避免把“奇数”和“素数”、“约数”和“质因数”混为一谈。

概念	意思	例子与边界
d 是 n 的约数	n 能被 d 整除	3 是 12 的约数，12 是 3 的倍数
素数（质数）	大于 1，正约数恰有 1 和自身	2 是唯一的偶素数
合数	大于 1，除 1 与自身还有其他正约数	9 是奇数，但不是素数
1	只有一个正约数	既不是素数，也不是合数
互质	两个正整数的最大公约数为 1	8 与 9 都不是素数，却互质

为什么试除只需到平方根?

如果 n=u×v，u、v 不可能都大于 √n。因子成对出现，至少有一个不大于 √n。**等于**平方根的那一格不能漏，例如 49。

边界要能解释
判断一个数是否为素数：先处理 n<2，再尝试可能的因子。对较大正整数，`d <= n/d` 可以避免 `d*d` 的乘法溢出。

奇偶性是便宜的第一层过滤

偶±偶为偶，奇±奇为偶，奇±偶为奇；乘积只有“两数都是奇数”时才是奇数。大于 3 的素数必为 6k±1，但反过来不成立，例如 25。

独立任务 T05
检查 1、2、9、49、53：给每个数分类，并用“定义或足够的试除”说明理由。再用一个数推翻“6k±1 一定是素数”。

回到原卷：2506 选5、选8；2509 选4；2603 判6。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

06.1　辗转相除不是魔法：把长方形里整块的部分拿走

有一块长 30、宽 18 的长方形地面，想用同样大的正方形砖铺满，不能切砖。砖的边长必须同时整除 30 和 18；想让砖最大，就是找最大公约数。

先从长的一边拿走一块 18×18 的正方形，还剩一块 12×18 的长方形。为什么可以只研究剩下的这块?

30 = 1×18 + 12

30×18 的整块 = 18×18 的正方形 + 12×18 的余块

原来能铺满整块的砖，既能分尽 30，也能分尽 18，所以也能分尽它们的差 12。反过来，能铺满 18×12 余块的砖，只要边长能分尽 18，也能铺那块 18×18 的正方形，拼起来仍能铺满原地面。

两块地面允许的砖边长完全相同，所以“最大允许边长”也相同。我们没有丢掉答案，只是把问题缩小了。

缩小过程，与变量变化逐行对应

```
int t = a % b;  // 先留下余块的另一条边
a = b;          // 下一轮的第一条边
b = t;          // 下一轮的第二条边
```

轮开始	先算余数 t	依次赋值后
a=30，b=18	t=12	a=18，b=12
a=18，b=12	t=6	a=12，b=6
a=12，b=6	t=0	a=6，b=0

为什么最后取 a，不取 b

b=0 表示没有余块需要再分。上一轮用边长6的正方形刚好铺完，因此留下来的 a=6 是答案。不能因为“最后算出了0”，就把0当最大公约数。

t 的作用是保住旧 a%b。若先写 a=b，再计算 a%b，左边的 a 已经变了，拿到的就不再是原来两数的余数。

接着往下读

下一页用同一规则区分 GCD 与 LCM，T06 再把算法放进“灯何时相遇”的场景。

06.2　GCD 与 LCM：铺方砖和再次相遇

> **这一节要弄懂什么**
> 能从现实含义区分最大公约数与最小公倍数，再追踪辗转相除中的两项变化。

两种问题，两种“共同”

长方形用同样大的正方形无缝铺满，最大边长对应 GCD；两盏灯分别每 a、b 分钟闪一次，同时闪后再次相遇的时间对应 LCM。

当前 a	当前 b	a%b	下一对
30	18	12	(18,12)
18	12	6	(12,6)
12	6	0	(6,0)

> **为什么余数能替换大数**
> 若 a=q×b+r，那么 a、b 的任何公约数也能整除 r；b、r 的公约数也能整除 a。所以 gcd(a,b)=gcd(b,r)。余数一直变小，直到为 0。

当采用“b==0 返回 a”的版本，最后一对是 (g,0)；若采用“a%b==0 返回 b”的版本，会早一层结束。**调用序列依代码终止条件而定。**

对正整数：gcd(a,b) × lcm(a,b) = a × b

实现时常先除再乘：`a/gcd(a,b)*b`，并选择足够宽的类型。先除能减小中间数，但结果过大仍会溢出；用 long long 也不是无限大。

辗转相除时间通常记为 O(log min(a,b))；循环版辅助空间 O(1)，递归版还要计同阶的调用栈。

> **独立任务 T06**
> 两盏灯分别每 12、18 分钟闪一次：用辗转相除求 GCD，再判断再次同时闪用哪个量。完整写出“b==0 结束”版本的参数变化。

回到原卷：2412 选5；2503 选4；2512 判2；2606 选4、判3。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

07.1 把 72 拆开：每除一次，到底数了什么？

72 可以写成 8×9，但 8 和 9 还不是质数，我们还没有拆到底。先尝试最小的质数 2：72 能除尽，剩36；36 还能除，剩18；18还能除，剩9；9终于不能被2整除了。

动作	剩余数	已经取出的因子
开始	72	无
除以2	36	2
再除以2	18	2、2
再除以2	9	2、2、2
换到3并连续除两次	1	2、2、2、3、3

每除一次，就从原数里取走一个因子。循环里的 n 会变小，所以它逐渐变成“还没拆完的部分”，而不是始终代表最初的 72。需要原数时，必须另外保存。

内层为什么是 while，不只是 if?

```
// 这里只展示处理一种因子 d 的局部动作
while (n % d == 0) {
    // 记录一个 d
    n /= d;
}
```

if 只除一次，72就只取走一个2，仍剩36。while继续检查，直到当前这种因子一个也取不出来，才能去找下一种。

> **“次数”与“种类”的计数位置不同**
> 数因子2出现几次，就在每次成功除2时加一；数不同质因数种类，则确认有这种因子时只加一次，再把它的所有副本除完。计数变量站在哪一层循环，决定它在数什么。

例如拆46，取走2后剩23。此时若试除条件已不满足，不能把23丢掉：它是最后剩下的一个质因子。反过来，如果余数已经是1，说明全部拆完，1不能再当一个质因子登记。

> **接着往下读**
> 下一页再把“种类、指数、所有约数的个数”分开整理，T07 用90检验。

07.2　质因数分解：种类、次数与约数个数

这一节要弄懂什么
先分清题目数的是“不同因子种类”还是“某个因子出现次数”。

把合数拆到每一块都是质数

$$72 = 2 \times 2 \times 2 \times 3 \times 3 = 2^3 \times 3^2$$

不同质因数有 2 种；质因子按重复计算共有 5 个；因子 2 的指数是 3。唯一分解定理说的是：**大于 1 的整数分解为质数乘积，忽略顺序后唯一**，允许重复。

试除时，剩余数也会变
找到一个因子 d 后要反复除，直到不能整除。每次除掉的是这一种因子的一个副本。试除结束后，若剩余数大于 1，它本身就是还没记录的质因子。

记录对象	72 的记录	常见混淆
不同质因数	{2,3}	不能看到一个 2 就计一种新因子
每个质因数的次数	2:3 次，3:2 次	不能只除一次就跳走
正约数个数	(3+1)(2+1)=12	质因数不是所有约数

约数为什么有这么多?

72 的约数可选 0、1、2、3 个因子 2，再选 0、1、2 个因子 3，共 4×3 种组合。枚举约数时 d 与 n/d 成对；完全平方数的平方根只算一次。

试除到平方根的质因数分解，按固定宽度整数运算计可给 O(√n) 上界。成功除因子的次数合计至多 O(log n)，不等于每个候选都除这么多次。

独立任务 T07
把 90 分解为质因数，分别给出不同质因数种类数、因子 3 的次数、正约数个数，并解释每个答案在数什么。

回到原卷：2409 选7；2412 选6、判3；2509 选9；2606 选8、判9。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

08.1 整圈走过多少不重要，我们先只记落在哪里

在纸上画6个位置，编号0、1、2、3、4、5，从0开始每次前进一步。走到5后再走一步，就回到0。走了17步，是2个整圈再多5步；走29步，是4个整圈再多5步。

17 = 2×6 + 5

29 = 4×6 + 5

两次都停在5；步数相差12，恰好是2个整圈

我们说17与29“模6同余”。它不是说这两个数相等，而是只看除以6留下的余数时，它们相同。记作17≡29 (mod 6)。

从同一位置继续做同样的动作

两人都在位置5，各自再走4步，都会停在3。因此同余的数同时加同一个数，仍然同余。也可以先把4步看成余数4，算(5+4)%6=3。

若把29步这段路完整走3次，总步数是87。每段包含的整圈不会改变最后位置，所以也可只把5步走3次：15步最终同样停在3。这就是乘法也能先取模的直观理由。

> **但不能把除法照样搬过来**
> 比如6÷2=3；若先模5，6只剩1，2仍是2，再做C++整数除法1/2却得到0。丢掉的整圈在“除成几份”时可能影响结果，不能把加乘的规则直接套给除法。

如果倒退一步，数学上落在位置5；C++的`-1 % 6`却给出−1。程序中的余数写法与我们希望的0到5编号并不完全相同，需要再把负余数调整回合法位置。

面对“两个数对哪些模数同余”，可以反过来看差：相差的步数必须恰好是若干整圈，也就是模数能整除它们的差。这样就把看似反复试余数的问题，变成了找约数。

> **接着往下读**
> 下一页整理运算关系与负数余数。T08换两个数，让你从“差包含几圈”找模数。

08.2　同余与模运算：余数相同，差能整除

这一节要弄懂什么
理解“模 m 同余”的含义，并知道取模能保留哪些运算关系。此项由考纲明确要求。

同一圈上的同一位置

把时钟看作长度为 m 的圆圈，走 a 步与走 b 步停在同一位置，就有 a≡b (mod m)。对非负整数，等价于 a%m=b%m；也等价于 m 整除 a−b。

29 与 17 模 6 同余：两者余 5，差为 12

运算（m>0）	可用关系
加法	(a+b) mod m = ((a mod m)+(b mod m)) mod m
减法	可先做余数相减，再调整到 0…m−1
乘法	可先取模再相乘取模，但中间乘积也要防溢出
除法	不能直接把普通除法照搬到模运算中

C++ 的负数余数
`-1 % 5` 为 −1；想把一个整数余数规范到 0…m−1，可使用 `((x % m) + m) % m`，前提是类型足够容纳中间计算。

数位和为什么能帮助判断被 9 整除?

10 模 9 余 1，100、1000 也一样。所以一个十进制数模 9 与各位数字之和模 9 相同。这个规则不能直接搬到“模任意数”。

独立任务 T08
35 与 11 对哪些大于 1 的正整数模数同余？请从“差”出发找全，再挑两个模数验算余数。

回到原卷：2512 选4；2509 判1；考纲第22页“同余与模运算”。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

09.1 一张素数表：划掉 6，与划掉 9，不是同一时刻

要知道 2 到 30 哪些是素数，可以逐个试除。但这样会反复问相似的问题：6能被2整除吗、8能被2整除吗……筛法干脆从2出发，一次把它的合数倍数划掉。

先把 0、1 排除；2开始暂时都标作“还没有发现它是合数”。请注意：**还没划掉，不等于在任何时刻都已经证明是素数。**要轮到这个数时才作判断。

按从小到大的顺序走

轮到的数	看到的状态	这轮干什么
2	没被划掉	确定2是素数；划4、6、8、10……
3	没被划掉	确定3是素数；从9起划9、12、15……
4	早被2划掉	已知是合数，跳过
5	没被划掉	确定5是素数；从25起划25、30
6	早被2划掉	已知是合数，跳过

为什么轮到3时，没被划掉就可以确定它是素数？如果它是合数，一定有一个比它小的质因子，前面那个质数本来就会划掉它。现在它还在，说明不存在这种因子。

再看内层两处看似很小的写法

处理3时，每次应该前进3格，才能走过9、12、15。若每次只加1，就会把10、11一起划掉，素数11也遭殃；若从3本身开始，则先把刚确认的素数自己划掉。

> **把12画两次，才看懂与线性筛的区别**
> 12在处理2时被划掉一次，处理3时又被划掉。重复写一次“合数”不影响判断正确，却仍然是一次工作。下一种筛法要解决的正是这些重复。

> **接着往下读**
> 下一页解释为什么起点可以跳到 i^2；再到线性筛时，请一直追踪12是由谁处理的。

09.2　埃氏筛：用素数划掉它的倍数

> **这一节要弄懂什么**
> 一次要找很多素数时，把“每个数分别试除”改成“从一个素数出发批量排除”。

在 2…30 的数表上做三件事

1. 最初把 2 起的数暂记为可能是素数，0、1 单独排除。
2. 遇到还没被标记的 i，它是素数；把 i 的合数倍数划掉。
3. 继续向后看，留下的就是素数。

素数 i	从 i^2 开始划掉	较小倍数在哪里处理了
2	4、6、8、10……	从 4 开始
3	9、12、15、18……	6 已经是 2 的倍数
5	25、30……	10、15、20 已有更小质因子

> **为什么从 i^2 开始**
> 小于 i^2 的合数倍数 $i\times k$ 满足 $2\leqslant k<i$，k 至少有一个更小的质因子，它先前已使这个数被标记。起点 i^2 减少重复，但埃氏筛仍可能重复标记某个合数。

```
for (long long j = 1LL * i * i; j <= n; j += i) {
    // 把 j 标记为合数
}
```

内层每次增加 i，才是在走 i 的倍数。若从 i 开始，会误删素数自己；从 2i 开始也可正确筛，但有更多重复。常见实现的时间为 O(n log log n)，辅助空间 O(n)。

> **独立任务 T09**
> 在 2…30 的表上分别圈出 2、3、5 的筛除轨迹，指出哪个合数被重复划到。解释 3 的内层为什么可从 9 开始。

回到原卷：2409 选5；2412 选7；2509 选6；2603 选6；2606 选6。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

10.1 线性筛的两个数：i 是当前数，p 是已知素数

先别背 break。线性筛里有两种角色：外层从小到大走过的当前数 i，以及已经发现的素数 p。我们把它们相乘，得到准备标记的合数。

例如 i=4、p=2，生成的是8；i=6、p=2，生成的是12。i 可以是素数，也可以是合数；p 才是从素数表里拿出的数。

前几轮真实发生了什么？设上界为20

当前 i	先看 i 的状态	试哪些 p	标记什么 / 为什么停
2	未标记，加入素数表[2]	2	标4；2能被2整除，停
3	未标记，表变[2,3]	2、3	标6、9；到3时整除，停
4	已经是合数，不加入表	2	标8；4能被2整除，停
5	未标记，表变[2,3,5]	2、3	标10、15；再乘5超20，停
6	已经是合数	2	标12；6能被2整除，停

观察4这一轮：它只乘2，不再乘3，所以12没有被4×3标记；12稍后交给6×2处理。两种乘法都能得到12，我们选了“其中素数因子最小”的那一种。

> **这张表帮助你区别三种停止**
> 素数表已经读完：没有下一个p；乘积越界：不能标记数组以外；i能被p整除：为了不重复，要主动停。三种原因负责不同的事，不能只留一个条件。

下一页才来解释最难的一点：为什么“i%p==0”恰好告诉我们，到这里应该停，而不是一个随便背下来的口令。

> **接着往下读**
> 保留这张前五轮记录，下一页用4×3和6×2解释停止规则。

10.2 为什么整除以后就停？从 4×3 和 6×2 看起

处理 i=4，先遇到素数2。因为4=2×2，4本身已经带着一个2。若继续让4乘更大的素数3，得到的12依然含2，它的最小质因子就不是这轮使用的3。

> 4×3 = (2×2)×3 = 2×(2×3) = 2×6
>
> **因此这件事留给“当前数6，素数2”来做**

一般地，若当前 i 能被 p 整除，i里面已经有p。再乘任何更大的素数q，乘积i×q里仍然含p。它应交给 p×(i÷p×q) 这个组合，不需要我们现在抢着标记。

为什么整除之前还可以继续?

素数表按从小到大排列。只要还没遇到能整除i的那个最小素数，当前p就小于i的任何质因子，于是i×p的最小质因子确实是p，这次标记正合适。

> **可以把合数看成“最小质因子 × 对应的商”**
> 每个合数都有唯一的最小质因子，而对应的商也随之确定。线性筛恰好让这一组来生成它，所以不会重复。注意是最小，不是最大质因子。

把含义放回代码，不再猜符号

```
if (i % p == 0) break;
```

左边问“当前数i是否已经含有这个素数p”；若答案是，是在本次乘积标记之后停止，不能在标记之前就跳走。否则像4=2×2这样的合数就漏掉了。

等你能用12解释这条规则，再记“每个合数只筛一次，所以O(n)”才有根。只背复杂度并不能帮助你发现写反的取余式。

> **接着往下读**
> 下面的整理页把角色、边界和复杂度合在一起；T10 用i=6再解释一次。

10.3 线性筛：让每个合数只登记一次

这一节要弄懂什么
读懂质数表、下标边界和 break 的分工。记住“最小质因子”背后的防重复理由。

依次处理 i=2…n。若 i 还未被标为合数，就加入素数表 primes。然后按从小到大的素数 p，标记 i×p。

```
for (int j = 0; j < primes.size(); ++j) {
    int p = primes[j];
    if (1LL * i * p > n) break;
    // 标记 i*p 为合数
    if (i % p == 0) break;
}
```

检查	保护什么
j < primes.size()	读 primes[j] 前，下标必须有效
i×p ⩽ n	被标记的数不能超出数表
i%p==0 后停止	只用不超过 i 的最小质因子的 p，防止重复筛

用 12 解释 break
i=4，p=2 时先标记 8，然后停止；不再用 p=3 标记 12。12 留给 i=6、p=2 处理。这样每个合数由它的最小质因子与对应商唯一生成。

标准线性筛为 O(n) 时间、O(n) 空间。O(n) 的理论增长慢于 O(n log log n)，但真实速度还受常数、内存访问和实现影响，不能断言“任何范围都更快”。

若用 is_prime，true 表示未排除；若用 is_composite，true 表示合数。读题先查布尔量含义，不能照抄真假值。

独立任务 T10
画出 i=6 时依次尝试哪些 p、标记哪些数、在哪里停止（设 n=30）。说明继续使用 p=3 会让哪个数重复。

回到原卷：2406 选7—8；2503 选6；2509 选7—8；2606 选5。先独立做，再说依据。

☐ 能解释变量角色 ☐ 能手推或举反例 ☐ 隔天换题仍会

11.1 把四级的“位置盒子”，接成一条链

还记得四级讲义里的两个盒子吗？普通变量保存数字，指针变量保存“另一个盒子在哪里”。链表没有换掉这个规则，它只是让一个结点里同时住着“数据格”和“位置格”。

A结点：[数据7 | 下一结点的位置B]
B结点：[数据4 | 下一结点的位置C]
C结点：[数据9 | 下一结点为空]

A、B、C 是为了画图方便给位置起的名字，不是结点数据。head 是另一个指针盒子，保存A的位置，让我们知道从哪里进入这条链。

沿箭头读代码

表达式	一步一步读	最后得到
head	读head这个位置盒子	A的位置
head->val	沿head到A，读A的数据格	7
head->next	沿head到A，读A的next位置格	B的位置
head->next->val	先到A，再沿A.next到B，读B.val	4

所以 `p->next` 本身还是一个位置，不是“下一个数字”。想取下一个结点的数据，还要再沿这个位置走过去。每走一次箭头，都先确认出发指针不是空。

> **这里最值得比较的两行**
> `p=p->next` 是把B的位置装进p，换的是p这个路标；`p->next=q` 是走到p指向的结点，改它内部的next位置格。后一行才会改变链表的路。

数组像连续编号的一排格子，可以通过起点与下标算出位置；链表每个结点都得告诉你下一站在哪。它们的逻辑顺序都能是7、4、9，但寻找第3个元素的方式不一样。

> **接着往下读**
> 下一页把“移动路标”和“改道路”并排比较；T11请画图证明哪些东西没变。

11.2　链表：数据盒子之间用箭头相连

这一节要弄懂什么
先画结点与指针，区分“移动指针”和“修改链接”。这是四级指针、结构体知识的继续。

```
struct Node {
    int val;
    Node* next;
};
```

head → [7 | next] → [4 | next] → [9 | nullptr]

写法	真实动作
p = p->next	让 p 这个指针指向下一结点；链表本身的链接没改
p->next = q	修改 p 所指结点里的箭头，使它指向 q
p->val	沿 p 找到结点，再读该结点的数据
new Node{5,nullptr}	创建结点：数据为 5，后继为空

数组与链表的代价

动作	数组	普通链表
访问第 k 项	O(1) 下标定位	通常 O(n) 沿链走
查找指定值（无其他结构）	最坏 O(n)	最坏 O(n)
已知合适位置后插入/删除	中间位置通常要移动 O(n) 项	改少量链接 O(1)
存储	元素连续，无额外链接域	结点可分散，需额外指针空间

先看题目给了什么
单链表仅知道待删结点指针，通常还缺前驱；已知前驱则容易删除。双链表可以沿 prev 找前驱。不要把“定位”和“改链接”的耗时混在一起。

独立任务 T11
为上面的三个结点分别画出 p=head、执行 p=p->next、再执行 p->val=6 后的图。说明 head 和各条 next 有没有改变。

回到原卷：2409 选1；2506 选1–2；2512 选3；2603 判1。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

12.1 先保住 Q 的位置，才能把 S 插进去

现在已有 P→Q→R，想在P后插入新结点S。这里的目标非常具体：P的下一站改成S，S的下一站是Q，Q到R原封不动。

开始时，我们能从哪里找到Q？从 `P->next`。这个位置格保存着Q唯一的入口，所以不能一开始就把它盖掉。

时刻	P.next存什么	S.next存什么	能否沿P走到Q
开始	Q	尚未接好	能
S先记住P的旧后继	Q	Q	能
P再改为指向S	S	Q	能：P→S→Q

```
S->next = P->next;
P->next = S;
```

把两行调换，再亲眼看错误发生在哪

若先做 `P->next=S`，P.next现在存的是S。随后做 `S->next=P->next`，抄到的也只能是S，于是S的箭头指回自己。

错误结果：P → S ↺　　Q → R 失去了从P进入的路

第二行在纸面上没改，但它读取的变量内容已经变了。这和四级里“先做a=b，再算a+b”不能继续使用旧a，是同一种问题。

> **遇到更长代码时的固定动作**
> 先画目标图，列出哪些链接必须成立；再按代码顺序，每执行一行就更新一个箭头。不要同时在脑子里把四行都当成已经完成。

双向链表只是在这件事上多了回去的路。插入后，S既要知道前驱，也要知道后继；两个邻居也要分别改掉指向对方的那条旧箭头。

> **接着往下读**
> 下一页扩展到双向链表与头尾边界。T12由你安排四条链接，不能只背刚才两行。

12.2　链表插入：接好新结点，再改旧入口

这一节要弄懂什么
理解赋值顺序。旧后继的位置一旦被覆盖，后半条链可能再也找不到。

单链表：在 P 后面插入 S

原来：P → Q → …
目标：P → S → Q → …

新结点 S 必须记住 Q；旧结点 P 再把后继交给 S。若先把 P 的后继改为 S，再从 P 的后继找 Q，此时看到的已经是 S。

双链表：两侧都要接上

原来：P ⇄ Q
目标：P ⇄ S ⇄ Q

需要成立的关系	检查方向
S.next 指向 Q，S.prev 指向 P	新结点知道左右邻居
P.next 指向 S，Q.prev 指向 S	两个旧邻居都认得新结点

边界有两类
普通双链表中 Q 可能为空，改 Q.prev 前要检查；若插在尾部还要更新 tail。循环链表可回到头，但要先确认题目带不带哨兵、空表怎样表示。

创建与头尾变化

空的普通链表首次插入：head 与 tail 都应指向新结点。头插时若需要让调用者的 head 改变，函数应返回新头，或用能修改调用者 head 的传参方式；仅给局部指针重新赋值不够。

独立任务 T12
在 A⇄B⇄C 中，把新结点 S 插到 B 前面。画目标图，并用四句“谁的哪个箭头改指向谁”安排顺序；再指出哪一步提前会丢失旧邻居。

回到原卷：2406 选4；2409 选2；2506 选3；2603 选2；2606 选1、判2。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

13.1 删除的对象，和正在站着的前驱，不是同一个人

假设要删去所有数据为2的结点，链表是 dummy→2→2→7。dummy是临时放在最前面的哑结点，它不算实际数据，只提供一个始终能站住的位置。

让cur站在dummy上。我们检查的是 `cur->next`，也就是cur的下一个结点是否要删除。cur自己的职责是“待删结点的前驱”。

一次删除有三个独立动作

```
Node* del = cur->next;    // 给待删结点单独留路标
cur->next = del->next;    // 前驱绕到它后面去
delete del;              // 链接接好后，才释放它
```

del是保存待删结点位置的指针；delete del释放的是它指向的结点。释放之后，不能再向那个结点问“你的next在哪”。所以这里的顺序不只是书写习惯。

时刻	cur站在哪里	cur.next现在是谁
删之前	dummy	第一个2
删第一个2之后	仍是dummy	第二个2
再删第二个2之后	仍是dummy	7
7不用删	才把cur移到7	空，结束

如果删一次就让cur前进一步，它会站到第二个2上。下一轮却检查cur.next，即7，于是第二个2被跳过了。**这不是少写一个判断，而是忘了cur一直是“前驱”。**

> **新头从哪里得到?**
> 原来第一个结点可能已经被释放，所以不能最后还返回旧head。dummy.next在每次接线后都指向当前第一个实际结点；全部删光时，它就是空。

> **接着往下读**
> 下一页把普通删除、双向删除和边界合起来看；T13再用5、5、8自行追踪。

13.2　链表删除：先绕过去，再释放

这一节要弄懂什么
删除的是结点占用的空间；释放后不能继续用这个结点找后继。

原来：前驱 P → 待删 D → 后继 Q
目标：P → Q，最后释放 D

先记住 D，并让 P.next 连到 Q，再释放 D。`delete` 不会替你把其他指针改好，也不会自动把指针变量设成 nullptr。

哑结点为什么有用?

dummy → [5] → [5] → [8] → nullptr

dummy 不存业务数据，却让“删第一个结点”和“删中间结点”都变成“删 cur 的下一个”。最终新头从 dummy.next 取得。

当前情况	cur 的角色与动作
cur->next 需要删除	cur 留在前驱，检查它新接上的下一个
cur->next 不需要删除	cur 才向后移
cur->next 为空	已经没有候选结点，结束

一个很小却很有用的反例
连续两个结点都该删除时，若每删一个就无条件移动 cur，可能跳过第二个。自测至少包含“连续待删、全部待删、空表”。

双链表删中间结点时，前驱与后继需要互相接上。删首尾还涉及 head、tail 或哨兵。**“拿后继覆盖当前数据”只是特殊技巧**，不适用于尾结点，也改变了结点身份语义。

独立任务 T13
在 dummy→[5]→[5]→[8] 中删除所有 5。每轮只标出 cur、待删结点、新链接，解释第一次删除后为什么 cur 不前进。

回到原卷：2412 选3；2509 选2；2603 选3；2606 选3。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

14.1 绕一圈以后停，和走到空以后停

普通链表有一个“没有下一站”的结尾，用nullptr表示。循环链表把最后一站接回起点，所以沿next走下去不会自然遇到空。

普通：A → B → C → 空

循环：A → B → C → A → B → C → …

遍历循环表，不是把普通表的while原封不动搬过来。我们要走完恰好一圈；停止的证据是“已经离开起点并重新回来了”。

时刻	p的位置	本轮处理后再前进
开始	A	处理A，到B
下一轮	B	处理B，到C
下一轮	C	处理C，到A
已经回到起点	A	结束，不再处理第二遍A

为什么常用do…while？因为开始时p本来就等于head。若直接写“只要p!=head才进入”，第一轮就无法开始。do…while先让我们处理一次，再检查是否绕回来了。

带哨兵时，谁才是实际数据?

若题目有一个不存实际数据的头哨兵H，形状可能是H→A→B→H。空表则是H→H。遍历实际数据时从H.next开始，遇到H停止，不能把H算作一件商品或一个人。

> **读链表题，先在纸角写三句话**
> 有无哨兵？head指向哨兵还是第一个实际结点？最后的next是空还是回到起点？这三句话没定清，就不要先背“空表条件”。

> **接着往下读**
> 循环改变的是终点。下一页再看另一种改变：把每条箭头反过来时，怎样保住后半段。

14.2 反转链表：当前箭头一转，前进的路要另存

拿最短的一条非空链 X→Y→空。我们想得到Y→X→空。只把head改为Y是不够的：Y的next还是空，X就被留在外面了。需要改变的是结点内部的箭头。

给三个角色起名字：prev指向已经反转好的部分，cur指向当前要处理的结点，nextSaved保存还没处理部分的入口。开始时还没有反转任何结点，所以prev为空。

第一轮（cur=X）	发生的动作	为什么
保存	nextSaved=Y	记住当前结点的旧后继
反向	X.next改为空（prev）	X成为已反转部分的末端
扩张	prev改指向X	已反转部分现在有X
前进	cur改指向Y	继续处理保存的后半段

第二轮，先记住Y的旧后继为空，再让Y.next指向prev，也就是X；随后prev指向Y，cur变为空。未处理部分已经没有结点，新的头就在prev里。

第一轮后：已反转 X→空 | 未处理 Y→空
第二轮后：已反转 Y→X→空 | 未处理为空

为什么保存必须早于改箭头
假如先让X.next指向空，再执行nextSaved=X.next，你保存下来的就是空；Y的位置已经被盖掉。和插入时丢掉Q，是同一个“先覆盖、后想用旧值”的错误。

这一过程不需要移动结点存储的位置，也不用交换数据，只修改next。每结点处理一次，为O(n)时间；额外只用几个位置变量，为O(1)辅助空间。

接着往下读
下一页归纳三种形态与快慢指针。T14换成三个结点，由你独立走完反转。

14.3 循环、反转与快慢指针：先说明停止的原因

这一节要弄懂什么
链表不同形态有不同终点；改变箭头方向时，要保住尚未处理的部分。

链表形态	尾部链接	典型遍历终点
普通单链表	next=nullptr	p==nullptr
循环单链表	next 回到起点	走过一轮后重新回到起点
带哨兵的循环单链表	可回到哨兵	p==哨兵；空表哨兵.next==自身

非空、无哨兵循环表可以先处理当前结点，再前进一步，回到 head 时停。开始时 p=head，如果一上来就要求 p!=head，反而一个也不会处理。

反转普通单链表：三个角色

变量角色	必须保留的东西
prev	已反转部分的新头，最初为空
cur	当前准备反向的结点
nextSaved	cur 原来的后继，防止断开后失联

一轮反转的动作
先保存原后继 → 当前箭头转向已反转部分 → 已反转部分向前扩一格 → 继续处理保存的后继。最后 cur 为空，prev 指向新头。此处针对普通单链表。

快慢指针判断环

slow 每次一步，fast 每次两步。有环时它们会相遇；无环时 fast 会遇到空指针。走两步前检查 `fast` 和 `fast->next` 均非空。不要在尚未移动时因起点相同就断定有环。

独立任务 T14
把 A→B→C→空反转，只画每轮开始和结束时 prev、cur、nextSaved 的位置。说明“先改 cur.next，再保存原后继”为什么会失败。

回到原卷：2512 选1；2606 选2；2509 选3；考纲第22页“反转”。先独立做，再说依据。

☐ 能解释变量角色 ☐ 能手推或举反例 ☐ 隔天换题仍会

15.1 二分的第一步：不是“算中点”，而是说明谁还可能

面前是升序的一排数：[1,4,7,10,13,16,19]，要找7的位置。先在下面写好下标0…6。数字7是目标值；下标2才是它的位置，别把两者混起来。

下标： 0 1 2 3 4 5 6

数值： 1 4 7 10 13 16 19

起初每个位置都有可能，记成[l,r]=[0,6]。这个版本包含两端，所以l和r的意思是“还没排除的最左、最右位置”。

检查中点以后，证据允许排除什么？

中点下标3，值10比7大。因为升序，3右边只会更大，也都不可能是7。我们可以一次排除下标3、4、5、6，把r改成2。

候选区间	中点位置 / 数值	根据证据做什么
[0,6]	3 / 10	7较小，保留[0,2]
[0,2]	1 / 4	7较大，保留[2,2]
[2,2]	2 / 7	找到，返回下标2

第二轮为什么l要变成mid+1，而不是mid？因为mid处的4已经证明不等于7，留着它没有用。第三轮为什么还要进入？因为[2,2]里仍有一个没检查的位置。

条件从区间含义长出来

闭区间[l,r]里，只要l≤r就还有候选；l>r才是空。这就是该写法使用while(l≤r)的依据，不是二分题永远都要背的固定符号。

如果目标不存在，候选也会被不断排除，最后变成空区间。算法结束不一定因为“找到”，也可能因为“已经没有地方可找”。必须同时考虑这两个停止原因。

接着往下读

下一页把找到与找不到的情况放进同一套规则。T15会换目标，让你独立追踪。

15.2　二分查找：每次排除不可能的一半

这一节要弄懂什么
先会找某个值，再学找边界；每种写法的区间、循环和更新必须成套理解。

约定：闭区间 [l,r]，找任意一个 x

数组已经升序排列，候选位置包含 l 与 r。中点 `mid=l+(r-l)/2` 取下中位。若 a[mid]==x，就找到；若 a[mid]<x，中点及其左侧都不可能；反之排除中点及右侧。

结果	新的候选区间	理由
a[mid]<x	[mid+1,r]	中点也已经被排除
a[mid]>x	[l,mid−1]	中点也已经被排除
a[mid]==x	返回 mid	本题只需一个位置

何时继续，何时失败
只剩一个候选（l==r）也要查，所以继续条件为 l≤r。若最终 l>r，区间空了，才是“没找到”。返回的是位置，不能拿比较次数当位置。

把过程记成四列

l	r	mid 与 a[mid]	下一步
0	6	3，值为 10	若目标是 13，应排除哪一半?

示例数组：[1,4,7,10,13,16,19]。原题若问“比较几次”，数检查了几个中点；若问 while 条件计算次数，还要留意最后一次失败的条件检查。

经典数组二分的时间 O(log n)，循环版辅助空间 O(1)。链表取中间位置不能按数组的 O(1) 随机访问计算。

独立任务 T15
在上述数组中分别找 13 与 12，列出比较过的值和区间。用 n=1 的数组再试一次，解释为何本版本不能把 l≤r 改成 l<r。

回到原卷：2406 选11；2409 选13；2412 选11、判9；2503 选11–12。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

16.1 已经遇到一个 4，为什么还不能立即返回?

现在数组换成[1,4,4,7,9]，题目不再问“随便找到一个4”，而是问“第一个大于等于4的位置”。下标1、2、3、4都满足值≥4，我们要最靠左的那一个。

先把每个位置是否满足条件写成一排：假、真、真、真、真。升序保证了一旦出现真，后面就不会再变假。我们找的是从假走到真的分界。

位置：0 1 2 3 4

条件：假 | 真 真 真 真

目标不是随便一处真，而是第一处真

中点满足了，仍可能就是答案

如果中点在下标2，它的值是4。它满足条件，但左边还可能有更早的4。于是继续向左缩；同时不能把下标2丢掉，因为“左边没有更早满足者”时，2本身仍是答案。

看到的情况	能确定什么	必须保留什么
a[mid]≥x	答案不会在mid右边	mid自己仍有可能
a[mid]<x	mid以及更左都不够大	只保留mid右边

同一句“相等”，换一个任务会改变动作

找任意等于值时，相等可以结束；找最左边界时，相等只是“这里可行”，还要向左验证有没有更早的位置。先读题目要什么，才决定相等时做什么。

这个思路也能找“不存在的x应该插入哪里”。例如x=6，虽然数组没有6，但第一个≥6的是7的位置。因此“返回一个下标”不自动表示找到了相等元素。

接着往下读

下一页把这条分界线装进l、r，并解释为什么r可以等于数组长度n。

16.2 把“没有满足元素”也给一个位置：n

沿用[1,4,4,7,9]，长度n=5。若找第一个≥12的元素，五个位置都不满足，应该怎样统一表示？约定返回5，也就是n，表示越过了最后一个元素。

这个5是返回标记，**不是可以读取a[5]的合法下标**。初值l=0、r=n，是把这个“没有”的答案也纳入考虑。

一次完整追踪：第一个≥4

l、r	mid	读到的值	下一步
0、5	2	4，满足	r=2，保留中点
0、2	1	4，满足	r=1，仍保留中点
0、1	0	1，不满足	l=1，排除中点
1、1	不再计算	候选答案只剩1	结束，返回1

为什么循环只在l<r时执行？因为此版本的目标是把答案范围挤到一个位置；l==r就已经确定。每次取下中位，mid<r；r≤n，所以读取的mid一定小于n，不会读a[n]。

> **用两个数就能查错**
> 设a=[1,4]、x=4，l=0、r=2，mid=1满足。正确写r=mid，接着检查左侧，最终返回1；若改成r=mid−1，会直接变成l=r=0，错误地返回值1所在的位置。

返回后若题目需要“等于x”，还要先检查l<n，再读a[l]比较。空数组同样成立：l=r=0，直接返回0，不进入循环，也不读取不存在的元素。

> **接着往下读**
> 下面的整理页再补“最后一个”的取中点差异。先用T16证明你理解了第一个的位置。

16.3　二分边界：中点还可能是答案，就保留它

这一节要弄懂什么
区分“找到就返回”和“找第一个/最后一个”。相等时也可能需要继续搜索。

找第一个 ≥x 的位置：答案可以是 n

常见写法初值 l=0、r=n，候选答案位置含 n 这个“所有元素都小于 x”的标记；实际可读元素在 [0,n)。循环只在 l<r 时继续，所以 mid<n，不读取 a[n]。

检查结果	缩小范围	中点是否可能是答案
a[mid]≥x	r=mid	可能，保留并向左继续找
a[mid]<x	l=mid+1	不可能，把它排除

退出时的 l 是什么
l==r，返回第一个 ≥x 的位置；若返回 n，说明没有这样的元素。若题目要第一个“等于 x”，还须检查 l<n 且 a[l]==x。空数组会直接返回 0=n，不能再访问 a[0]。

找最后一个可行位置：警惕两格时不动

采用闭区间、`l<r` 且可行时写 `l=mid` 的版本，常需取上中位 `l+(r-l+1)/2`。只剩 [2,3] 时，下中位为 2，l=mid 会原地不动。

若使用“记录 ans，再 mid±1”的另一套写法，规则不同。**理解区间再选模板，不把不同版本的语句拼在一起。**

独立任务 T16
对 [2,5,5,5,9] 分别查第一个 ≥5、≥6、≥10 的位置，手推其中一次。再用两格区间解释上中位解决了什么问题。

回到原卷：2412 选12；2506 选11；2512 选11；2603 选8；2606 选9。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

17.1 二分答案先做检查题：给你 x，能不能办到？

有一根长10的木头，最多切2刀，希望每段都不超过某个长度x，并让x尽可能小。先别写二分，我们只回答一个更简单的问题：如果x已经给定，能否做到？

例如x=4，可以切成4、4、2，共3段，切2刀，能做到。x=3则至少要分成3、3、3、1，共4段，需要3刀，超过限制。

允许最长 x	至少要分成几段	至少几刀	最多2刀能办到吗
2	5	4	不能
3	4	3	不能
4	3	2	能
5	2	1	能
10	1	0	能

这里的check(x)就负责最后一列。它不负责“找最小”，只判断当前长度限制是否可行。把这两份职责分开，外层二分才有一个可靠的判断工具。

刀数为什么不是段数？

开始就有1段。每切一刀，总段数增加1，所以k刀最多分成k+1段。长度L需要的最少段数是向上取整L/x，刀数再减1。

> **不背公式，先分能否整除**
> 若L恰好等于q×x，需要q段、q−1刀；若还多出一点，需要q+1段、q刀。对正整数，两种情况统一成整数除法(L−1)/x。

如果有多根木头，每根都从“原本一段”开始，各自算刀数后相加。不能先把所有长度相加，当作已经接成一根木头。

> **接着往下读**
> check能说清后，下一页才引入二分；先说明x变大，究竟让要求变严还是变松。

17.2 答案没有写在数组里，也能找“第一处可行”

继续上页的木头问题。x越大，允许每段越长，要求越宽松：原来按4能完成的切法，在允许5时当然也能用。因此可行性是一段“不能”，后面接一段“能”，不会来回反复。

候选x：1 2 3 | 4 5 6 7 8 9 10

可行性：假 假 假 | 真 真 真 真 真 真 真

这和“找第一个≥x的位置”很像，不过现在候选是长度值，判断依据是check。我们找的是第一处真，也就是最小可行答案。

l、r	尝试mid	check的结果	怎样保留答案
1、10	5	可行	5可能最小，保留它：r=5
1、5	3	不可行	3及更小都不行：l=4
4、5	4	可行	保留4：r=4
4、4	不用再猜	范围只剩4	答案确定

换一种木头题，可行方向可能反过来

如果改问“切出至少m段等长木段，段长最大多少”，x越长能得到的段数越少，要求更难。状态就变成“能…能，不能…不能”，目标是最后一处真。

> **两个模型都叫切木头，更新方向却不同**
> 不要看故事名字选模板。先说清check(x)的含义，再判断“如果这个x可行，更大的x是否一定可行”。这句话的答案决定了保留哪一半。

还要把一次检查的工作计入复杂度。如果每猜一个x都要扫n根木头，那每轮就花O(n)；二分轮数是O(log D)时，总检查工作是O(n log D)，不能只写O(log D)。

> **接着往下读**
> 下一页把最小可行、最大可行与总工作量整理在一起；T17用长度11独立设计检查。

17.3　二分答案：猜一个要求，检查做不做得到

这一节要弄懂什么
有时二分的不是数组下标，而是答案大小。必须先证明可行性只会朝一个方向变化。

最小可行：最多 K 刀，让每段不超过 x

木头长 L，x 越大，要求越宽松。长度为 L 的一根木头最少切 `ceil(L/x)-1=(L-1)/x` 刀（正整数整除）。多根木头把所需刀数相加。

小 x：不可行　不可行　|　可行　可行：大 x
目标：找到第一处“可行”

可行时 mid 仍可能是最小答案，因此保留它并向小处找；不可行则排除它及更小值。

最大可行：至少 m 段等长木段

x 越长，可得到的段数 Σ `floor(Lᵢ/x)` 越少；可行状态是“真…真假…假”。求的是最后一处真，更新方向相反。

先写 check 的一句话
check(x) 到底检验“至少几个”“最多几刀”还是“最小间距”？明确 x 变大后条件变严还是变松，才能决定朝哪里找。

在已排序坐标中放 k 个物品，要求间距至少 d，d 变大更难；可用“每次选最早可放位置”做检查。这是二分与贪心配合，并不互相排斥。

总时间 = 预处理 + 二分轮数 × 一次 check 的工作量

例如排序 O(n log n) 加每次 O(n) 的检查，共 O(n log n+n log D)，D 为搜索的数值跨度。实数二分（如求平方根）常以 r−l 小于精度 ε 停止，每次用中点替换端点；不能照搬整数版的 mid±1。

独立任务 T17
一根长 11 的木头，最多切 2 刀。分别检查 x=3、4、5，画出可行性方向，再说明如何寻找最小可行 x。“2 刀”与“2 段”是否是同一个限制?

回到原卷：2509 选10；2512 选12；2603 选7、选10、判7；2606 选10。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

18.1 “分别解决”以后，怎样把答案接回来?

找[3,8,2,6]的最大值，可以分成左边[3,8]和右边[2,6]。左边交回8，右边交回6，再比较这两个数，就能确定全体的最大值。

这个方法成立，是因为整体最大值一定在某一边，而那一边的最大值已经代表了它。合并时，我们不必重新查看所有元素。

原任务：max([3,8,2,6])

分成：max([3,8]) 与 max([2,6])

分别交回：8 与 6 → 合并得到 8

换成连续子段，就多出一种可能

现在问[−3,5,2,−4]中，哪一段连续元素的和最大。在5和2之间分开后，左半内部最好是5，右半内部最好是2。但是只比较5和2，会漏掉跨过分界的[5,2]，它的和是7。

答案的位置	子任务能直接提供吗?	本层还要做什么
完全在左边	能，左半的最佳连续段	收下左边的答案
完全在右边	能，右半的最佳连续段	收下右边的答案
跨过分界	不能，两边都只管自己内部	单独求跨界候选

跨界段必须紧挨着分界：左侧取一段以5结尾的后缀，右侧取一段从2开始的前缀。左侧不能跳过5去挑−3，右侧也不能跳过2去挑−4，否则就不再连续。

> **先问合并需要什么信息，再决定子任务怎么交回**
> 分成两半并不是分治的全部。要检查：原问题的每一种答案位置，是否都被左右子问题或本层合并覆盖。漏掉跨界这一类，再正确的递归也补不回来。

复杂度同样看“合并”做多少事。找最大值每次只比较两个结果；这里若每层都扫描左右边界来找前后缀，则各层合计工作不同，不能因为都分两半就写成相同复杂度。

> **接着往下读**
> 下一页用另一组数由你独立检验三类位置，并检查全负数组的边界。

18.2　分治：分开求，最后必须接得起来

这一节要弄懂什么
分治是一种组织解题的方法；递归是实现方式。同一个算法可以同时属于两者。

三个动作

步骤	找数组最大值	排序
分	分成左右两段	分成左右两段
治	分别求两段最大值	分别把两段排好
合	比较左右最大值	合并两个有序段

不是所有“取最大值”都叫贪心
把左右子问题都完整解决，再比较结果，是分治；没有做一个“选了就不回头”的局部选择。用循环维护当前最大值则是迭代实现。

最大连续子段和：合并时不能漏“跨中点”

最佳连续段可能全在左边、全在右边，也可能跨过中点。跨中点时，应拼接“左半段以中点结尾的最佳后缀”和“右半段从中点后开始的最佳前缀”。

[−2, 4 | 3, −1]

左内最佳、右内最佳、跨界最佳，要分别考虑

如果只取左右最大值，就会漏跨界情况。若不允许空段，全负数组也必须选一个实际元素，不能把所有最优值随手初始化为 0。

每层扫描合计 O(n)、约 log n 层时，总时间 O(n log n)；分治找最大值的合并每结点仅 O(1)，总时间则是 O(n)。

独立任务 T18
对 [−2,4,3,−1] 计算三种位置的最佳连续段；再用 [−5,−2] 检查“答案先设为 0”的风险。先说模型，不写完整代码。

回到原卷：2503 选14；2506 选9−10；2509 选14；2603 选11。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

19.1 两个队首，三个位置：把合并完整走一遍

左队[1,4,8]、右队[2,3,9]都已经按从小到大排好。要选所有剩余数字中的最小值，只需比较两个队首：每队后面的数都不会比自己的队首更小。

先给位置分工：i是左队下一个未取的位置，j是右队下一个未取的位置，k是结果数组下一个空位。三者初值都是0，但代表三件不同的事。

取出前 i、j、k	比较 / 操作	写进结果	取出后 i、j、k
0、0、0	1<2，取左	c[0]=1	1、0、1
1、0、1	4>2，取右	c[1]=2	1、1、2
1、1、2	4>3，取右	c[2]=3	1、2、3
1、2、3	4<9，取左	c[3]=4	2、2、4
2、2、4	8<9，取左	c[4]=8	3、2、5
3、2、5	左队已空，补右队	c[5]=9	3、3、6

第5次比较以后，i=3已经等于左队长度，不能再读L[i]。但右队还有9，因此“不能继续比较”不等于“合并完成”。收尾循环负责搬走剩余元素。

> **一次取走一个，才不会丢元素**
> 取左边，只推进i；取右边，只推进j；无论取哪边都推进k。若每轮让i、j同时前进，就把没有被选走的那一项跳过了。

为什么先排好左右两队很关键?

假如左队是[4,1]，右队是[2,3]，比较队首会先取2，却没有看到藏在4后面的1。合并能只看队首，依赖“左右各自有序”，这正是递归子任务先完成的工作。

这里的i、j属于两份临时数组；若代码直接用原数组的左右区间，初值与边界会改变，但“未取队首”的职责不变。读代码时先找职责，再认下标写法。

> **接着往下读**
> 下一页把相等元素贴上身份标签，由你检查合并是否保留了原来的先后关系。

19.2　归并排序：先让两队有序，再比较队首

这一节要弄懂什么
读懂 i、j、k 三个位置角色，以及为什么必须补上未耗尽的一边。

左队 L：[2, 7, 9]　　右队 R：[3, 5, 8]
i 看左队未取的第一个，j 看右队未取的第一个，k 指输出位置

左右都还有元素时，比较队首；取较小的那个，只移动它所在一边的指针，输出位置也移动。当一边空了，另一边剩余元素已然有序，直接追加。

当前左队首	当前右队首	取谁	哪个指针动
2	3	2	i 与 k
7	3	3	j 与 k
7	5	5	j 与 k
7	8	由你继续	

两种下标体系别串用
若用原数组闭区间 [l,mid] 与 [mid+1,r]，条件是 i≤mid、j≤r；若复制到长为 n1、n2 的临时数组，条件是 i<n1、j<n2。比较的是值，不是 i 与 j 谁大。

稳定性藏在相等时的处理里

两个元素关键字相等，仍保持原先前后顺序，叫稳定。归并时相等优先取左边，可保持稳定。把人贴上 A、B 标签，只按分数排序，才能看出谁被越过了。

通常数组归并需要 O(n) 临时空间；标准实现每层搬运 O(n)，总体 O(n log n)。

迁移到逆序对：若左右已分别有序且 a[i]>a[j]，左边从 i 到 mid 的每个值都与右边 a[j] 构成跨区间逆序，新增 mid−i+1 对。完整算法还须递归统计两侧内部并完成合并，单个计数片段不够。

独立任务 T19
把 L=[2A,5]、R=[2B,4] 合并（A 原来在 B 前，按数字排序）。列出 i、j 的推进，说明相等时取右边会改变什么。

回到原卷：2406 选13；2509 选12；2603 选12；2606 判4。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

20.1 先保存基准，再看两端怎样交换

用[4,2,5,1,3]演示一种具体的划分：首元素作基准，先从右扫描。下标从0开始；pivot保存数值4，i=0，j=4。所有扫描都要求i<j。

右指针越过≥4的值，停在一个<4的值；左指针越过≤4的值，停在一个>4的值。若还未相遇，就交换这两个放反了位置的数。

阶段	i、j 的变化	数组 / 说明
右边先找小值	j=4，值3<4，停	[4,2,5,1,3]
左边再找大值	i从0走到2，值5>4，停	4、2已越过；5停住
交换放反的数	i=2、j=4尚未相遇	[4,2,3,1,5]
再次右扫描	j从4退到3，值1<4，停	右侧的5已在合适一边
再次左扫描	i从2走到3，与j相遇	仍是[4,2,3,1,5]
基准归位	交换a[0]与a[3]	[1,2,3,4,5]，4归位

这一次碰巧全有序，不代表每次都这样

换成[4,3,2,1,5]，同样的规则相遇在下标3，归位后是[1,3,2,4,5]。基准4已经在正确位置，但左侧3、2还没排好，要继续递归处理。

> **这一轮真正保证了什么?**
> 这一版本保证：归位后的基准左侧不大于它，右侧不小于它。它没有承诺两边内部也有序。递归时排除已归位的基准，去处理左右剩余区间。

为什么首元素版本先从右找？在这套“相遇后把首基准换过去”的规则里，要让相遇处的数适合换回左端。若随意改成先左后右，[1,2]就可能在下标1相遇并把1、2换反。扫描顺序与最后交换是一整套约定。

> **接着往下读**
> 下一页对照两种划分写法；T20换一组数，只要求解释第一轮，不急着把整份快排写出来。

20.2 快速排序：一轮划分，不等于排完

这一节要弄懂什么
明确基准值 pivot 与指针职责。每种 partition 的边界和扫描顺序必须结合具体代码。

先建立共同目标

快速排序按基准把元素划到两边，再分别递归处理较小区间。一轮结束后，两侧内部通常还没有完全有序。

双向扫描的一种常见版本

以首元素为基准，右指针先找小于基准的值，左指针再找大于基准的值，未相遇时交换。相遇后再把首位置的基准交换到相遇位置。每次移动都带上 `i<j` 的保护。

为什么不能随便调换两次扫描
在“首元素作基准、相遇后与首元素交换”的这套代码里，扫描次序参与保证相遇位置合适。先左后右并不是逐字替换就能保持正确。试 [1,2] 即可发现问题。

另一种：单向扫描维护分区边界

以末元素为基准，i 表示“小于基准区”的末尾，j 遍历尚未分类的元素。找到小值时扩张小值区，再交换进去。最后基准放在 i+1。

角色	必须说清
pivot	是基准的值，不一定一直在 mid 位置
i、j	是扫描位置，还是某一分区边界?
递归范围	哪些元素已经就位，不应再包含?

Hoare 类划分、首元素归位版、末元素归位版并非同一套边界。不要把某题的 i≤j、另一题的 i<j 和递归区间混用。

独立任务 T20
用“首元素作基准、先从右扫描”的规则手推 [3,1,4,2] 的第一轮，只做到基准归位。解释为什么还需递归。

回到原卷：2406 选9；2412 选9；2503 选10；2509 选11；2606 选11。先独立做，再说依据。

☐ 能解释变量角色 ☐ 能手推或举反例 ☐ 隔天换题仍会

21.1　快排为何退化：划分是否平衡很重要

> **这一节要弄懂什么**
> 区分“平均很快”“最坏保证”“稳定性”。名字里有“快”不能作为推理依据。

两棵递归树，成本不同

划分形态	每层与层数	结果
大致分成两半	每层合计 O(n)，约 log n 层	平均时间 O(n log n)
每次分成 0 和 n−1	n+(n−1)+…+1	最坏时间 $O(n^2)$

有序数组配合“总选首元素”为基准，容易出现第二种形态。选中间位置不代表选中了数值中位数；随机基准或三数取中可改善常见表现，但不是消除最坏情况的保证。

稳定性是另一件事

算法快慢讨论工作量，稳定性讨论相等关键字的先后顺序。改变选基准方式不会自动让普通快速排序变稳定。

> **一个可画的稳定性反例**
> 按值排序 [2A,2B,1]。常见末元素划分把 1 与 2A 交换后可能得到 [1,2B,2A]；相等的两个 2 已经反序。普通快速排序不稳定。

递归栈也随划分变化：平均 O(log n)，最坏 O(n)。因此“划分原地进行”不等于整个递归算法额外空间严格为 O(1)。

> **独立任务 T21**
> 对升序 [1,2,3,4,5] 总选首元素作基准，画每轮剩余子问题长度。然后解释“已排好，所以这份快排只需 O(n)”为什么不成立。

回到原卷：2409 选9；2512 选10、判6；2603 选13、判3。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

22.1 七次合并，为什么仍要数三层搬运?

把8个单元素组归并成一组。先两两合成4组，再合成2组，最后合成1组。为了看清工作量，分别数“调用合并几次”和“每次要搬多少个元素”。

合并层	合并调用	每次输出元素数	这一层输出总量
第1层：单元素→2个一组	4次	2	4×2=8
第2层：2个一组→4个一组	2次	4	2×4=8
第3层：4个一组→8个一组	1次	8	1×8=8
累计	7次	各次长度不相同	24次元素输出

7次合并，是8组变成1组所需要的合并次数；24次元素输出，是按“每次向合并结果写入一个元素算一步”统计的工作。若实现还要复制回原数组，实际搬运会增加，但量级不变。

把8换成16，合并层数从3变成4，每层总输出也从8变成16。这就是“每层n，层数$\log_2 n$”相乘的来历。不是看到两层循环，就凭外形猜n^2。

比较次数又是另一本账

合并[1,2]和[3,4]，只需比较两次：取出1、2后左队空，3、4直接追加。合并[1,3]和[2,4]则要比较三次，直到取出3才有一边空。

> **长度p、q都大于0时，最坏比较p+q−1次**
> 每次比较至少取走一个元素；当已取走p+q−1个，只剩最后一个，不必比较便知道它应追加。交错排列能让两边尽量晚耗尽，从而达到这个上限。

因此面对同一段归并代码，题目问调用次数、元素搬运量、大小比较次数，会得到不同答案。先写清楚自己要数的动作，再画过程。

> **接着往下读**
> 下一页把这几本账放在同一张表中，T22由你构造一个达到最坏比较次数的例子。

22.2　排序核对表：次数、时间、稳定性分别算

这一节要弄懂什么
把容易背混的性质放在同一张表，再用结构解释 merge 调用次数。

通常实现	最好时间	平均 / 最坏时间	稳定性
冒泡（有提前结束）	O(n)	$O(n^2)$ / $O(n^2)$	稳定
插入（相等不跨越）	O(n)	$O(n^2)$ / $O(n^2)$	稳定
简单选择排序	$O(n^2)$	$O(n^2)$ / $O(n^2)$	不稳定
标准数组归并	O(n log n)	O(n log n) / O(n log n)	相等先取左时稳定
普通快速排序	O(n log n)	O(n log n) / $O(n^2)$	不稳定

表述针对常见实现。没有提前结束的冒泡，即使已排序也可能仍做平方级比较；稳定性也要结合相等时的比较和交换。

merge 调用次数为什么是 n−1?

标准归并排序把 n 个元素分到每组一个，再两组合成一组。开始 n 组，每合并一次组数减少 1，最后只剩 1 组，所以总共 n−1 次合并（n≥1）。

题目问什么	应数什么
合并 n 个单元素组到一组	n−1 次 merge
两队长度 p、q 的最坏比较次数	p+q−1（p,q>0）
标准归并整个排序的时间	每层 O(n) × 约 log n 层
递归调用总数（含叶子）	标准二叉拆分树共 2n−1 个节点

会迁移的区分句
“merge 只被调用 n−1 次”不代表总时间 O(n)，因为不同调用的合并区间长度不同。

独立任务 T22
将 5 个单元素组归并为一组，数合并次数。再给出两队各 3 个元素的例子，让归并比较次数达到最坏值，并手推证明。

回到原卷：2409 选10；2506 判4−5；2603 选14；2606 选13、判10。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

23.1　贪心：当前最优，需要一个“不吃亏”的理由

这一节要弄懂什么
能辨认贪心思想，也能判断一个贪心规则是否有证据支持，二者不是同一个问题。

贪心在做什么？

每步按一个规则挑当前最有利的选择，并把它固定下来，通常不回头枚举所有方案。例如按最便宜优先买商品、按最早结束安排活动。

局部好，不保证最终好
硬币面额 1、3、4，凑 6：每次拿最大能拿的面额会得到 4+1+1；但 3+3 更少。所以“有 1 元硬币”并不足以保证大面额优先最优。

证明常用的想法：交换

假设某个最优方案第一步与我不同，能否把它换成我的选择，而不减少最终能完成的目标？若能，固定第一步后再处理剩余问题。

最优子结构：整体最优解包含适当子问题的最优解。它有用，但单凭这一条不保证贪心成立；还要证明当前贪心选择可以安全采用。

相似词	真正区别
贪心	按规则作出局部选择，并固定
分治	子问题分别求解，再合成原问题
递归 / 迭代	实现过程的方式，不直接等于某种最优性策略

读到“一定”“所有”“任意情况都最优”，先找一个最小反例；反例足以推翻，几个成功样例却不足以证明普遍正确。

独立任务 T23
面额 1、4、6，凑 8。分别按“最大面额优先”和你能想到的更好组合完成，判断这条规则能否保证最少枚数。

回到原卷：2406 选2、判6；2506 选13；2603 判8；2606 判6。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

24.1 活动选择：结束得早，为什么值得先选?

同一个场地安排活动，每场活动价值都算1，只求能参加的场次最多。允许上一场结束时立刻开始下一场。三个活动是A=[1,7]、B=[2,3]、C=[3,5]。

如果按开始早先选A，场地一直被占到7，只能选1场。若先选结束最早的B，之后还能选C，共2场。开始早与结束早，在这个目标下不是同一个标准。

按结束时间扫描	当前lastEnd	能否选入	选后lastEnd
B=[2,3]	尚未选择	能，选B	3
C=[3,5]	3	3≥3，能接上	5
A=[1,7]	5	1<5，冲突	仍是5

lastEnd的职责是“上一个已选活动的结束时间”。看到一个没选的活动，不应更新它；否则就把一个根本没参加的活动当成了自己的安排。

一个成功例子还不是证明

设某个最优方案的第一场是X，我们选的最早结束活动是E。E结束不会晚于X。把方案第一场X换成E后，原来能接在X后面的活动，仍然能接在E后面；场次没有减少。

这说明存在一个同样好的最优方案，它的第一步与我们一致。固定E后，对它结束之后的剩余活动重复同样的推理，选择就有了依据。

> **证明使用了题目的哪些条件?**
> 每场价值相同，目标是场次最多；活动占用区间已知；可行性只取决于是否重叠。如果每场奖金不同，最早结束不一定能让总奖金最多，必须重新分析目标。

把饼干分给不同胃口的孩子也类似：最小的够用饼干先满足最小胃口，大饼干留给更大胃口。这里比较的是“能满足多少人”，并不是让某一个人拿到最大的饼干。

> **接着往下读**
> 下一页看另一种“留机会”：最重与最轻配对时，为什么不会浪费一个盒子。

24.2 两件一盒：最轻也配不上，就不再找了

每盒最多装两件，限重8，每件本身都不超过8。先按重量排成[1,2,4,5,7]。左指针l指最轻的未装物品，右指针r指最重的未装物品。

未安排的重量	检查	这一盒	下一轮
1,2,4,5,7	1+7≤8	1和7	l右移，r左移
2,4,5	2+5≤8	2和5	l右移，r左移
4	只剩一件	4单独一盒	处理结束

若当前最轻是2，最重是7，和已超过8，那么其余候选都不比2轻，当然更配不上7。7只能单独装；这时只移动右指针，2还没有被用掉。

能配最轻时，为什么可以放心配?

设最重为H、最轻为L，且两者能同盒。拿一个最优方案来比较：如果它们已经同盒，就不必改。

若H与X同盒，L与Y同盒，把它们换成(H,L)和(X,Y)。因为Y≤H，而原来H+X没有超重，所以X+Y也不会超重，盒子数没有增加。

若其中一件原来单独装，把H、L合在一起，将被换出的那件单独装，也不会增加盒子。若两件本来都单独装，合在一起还能省一个盒子。因此总能得到一个不更差、且H与L同盒的安排。

> **指针从“物品是否已经用掉”决定**
> 配成一对，两件都已处理，左右都动；最重单独装，只有右边被处理，左边不动。最后l==r时还有一件，不能把“只剩一件”误当成“已经没有了”。

整个判断利用了“每盒最多两件”。如果改成三件一盒，刚才交换两盒物品的证明就不再直接适用。读到类似故事，先核对数量限制。

> **接着往下读**
> 下一页再看“最晚空位”的含义；本专题的独立任务会换一组重量验证双指针。

24.3 有截止时间的任务：为什么把赚钱的事往后放?

三项任务都恰好花1个时间单位，截止时间表示必须在哪个时槽结束前做完。A截止2、收益60；B截止1、收益50；C截止2、收益40。每个时槽只能做一项，目标是总收益最多。

先按收益从高到低考虑。A最赚钱，但它可以放时槽1或2。如果随手占了1，接着就没法做必须在1完成的B。把A放在最晚可用的2，时槽1就留给B。

考虑的任务	截止前最晚空位	安排后的时槽1 / 2	累计收益
A：截止2，收益60	2	空 / A	60
B：截止1，收益50	1	B / A	110
C：截止2，收益40	没有空位	B / A	110

“先选收益高”与“尽早执行”是两回事

排序顺序是决定先考虑谁；时槽是决定它实际何时执行。A先被考虑，最后却在B后面执行，这两者并不冲突。

在不超过截止时间的前提下，任务往后挪，早时槽就能留给截止更早的任务。收益高者优先，则尽量让有限的可行位置用于更有价值的任务。

> **这条规则的适用前提要一起记**
> 各任务耗时都是1；收益非负；任务可自由选择且没有前置依赖。若任务耗时不同，收益高者优先不再自动最优：耗时2收益60的任务，可能挡住两项各耗时1收益40的任务。

这与活动选择的排序依据不同：活动选择求场次，先看结束时间；此处求收益，先按收益考虑，再安排时槽。两道题都“先排序”，不等于它们有同一个贪心规则。

本页先建立识别模型与手推的能力。正确性完整证明还需说明可行任务集合的交换关系；不能只用这三个任务的成功结果代替普遍证明。

> **接着往下读**
> 下一页将四种模型并排核对；先说清目标和前提，再选择规则。

24.4　四类贪心：每次保留后面的机会

这一节要弄懂什么

把经典模型与它们成立的条件对应起来；同样是排序，依据可以完全不同。

问题与前提	常用选择	为什么有希望不吃亏
最多不重叠活动；收益都按一个活动计	优先结束早的；下一场开始≥上一场结束	尽早腾出场地，给后面留下更多时间
最少两人船 / 两件盒；每人每件能单独装下	最重先安排；能配最轻就配，否则独坐	最轻都配不上，其他人更不可能配上
最多满足孩子；每人至多一块饼干	小胃口配最小够用的饼干	把更大的饼干留给更大的胃口
单位耗时任务，截止时间与非负利润	利润高者先考虑，放入截止前最晚空位	把早时槽留给截止更早的任务

必须连接到变量角色

活动题 lastEnd 是“上一个已选活动的结束”，不是随便改输入；双指针装箱时，成功配对消耗两件，失败只消耗最重件；最后只剩一件也要一个盒子。

先找能区分策略的小例子

“最早开始”与“最早结束”不同。例如 [1,8]、[2,3]、[3,4]。开始最早的一场会占很久，不一定留下最多场次。

只有“每件最多两人/两件”等条件成立时才套相应模型；若允许三人一船，不能把同一证明照搬。最重与最轻的配对，完整证明可用交换两条船的乘客方式完成。

独立任务 T24

重量 [2,3,3,4]，每盒限重 6 且最多 2 件。画每个盒子的安排和双指针变化。解释 4 为什么先尝试和 2 配对，以及只剩一件时怎样计数。

回到原卷：2412 选13；2509 选13；2512 选14；2606 选14。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

25.1 一格装一个数位：进位是一份交给下一格的十

想象普通整数变量是一只容量固定的盒子，数太大时会装不下。高精度把一个大数拆成很多格，每格保存一个十进制数位。格数可以增加，但每格的含义必须先定好。

以376为例，低位在前保存为[6,7,3]。下标0是个位，1是十位，2是百位。这三个数表示6+7×10+3×100；数组写法反过来，不是把376改成了673。

把376与58逐位相加

本位	两个数的本位值	旧进位	总和t	本位留下 / 新进位
个位 i=0	6与8	0	14	4 / 1
十位 i=1	7与5	1	13	3 / 1
百位 i=2	3与0	1	4	4 / 0

个位的14个“一”，是4个“一”和1个“十”。所以本格留4，下一格收到进位1。到了十位，总和13表示13个“十”，应留下3个“十”，再交给百位1个“百”。

结果按低位在前是[4,3,4]，输出时从高位倒着读，得到434。较短的58没有百位，把这一位当0；不能因为它先结束就漏掉376剩下的位。

```
t = a_digit + b_digit + carry;  // 先用旧进位算总和
result_digit = t % 10;          // 本位留下的数
carry = t / 10;                 // 交给下一位的十
```

三行次序的意思是“先结清本位，再交接”。如果先覆盖carry，再拿它参与本位总和，就把旧进位弄丢了。读局部填空时，先找谁是旧状态，谁是新状态。

全部输入位处理完，还要看进位

如95+7，个位留下2并进1，十位产生10又进1。两数的原有位都处理完后，仍需增加百位1。结果数组应为最高位进位留出空间。

接着往下读

下一页整理存储约定、字符转数值和位数；T25换数后独立填写每轮记录。

25.2　高精度与加法：数组保存每一位

这一节要弄懂什么
数字大到普通整数装不下时，改用多格保存数位；这是整数的位数问题，不是换成 double。

先登记“个位在哪里”

数 507：低位在前 [7,0,5]；高位在前 [5,0,7]

数组下标是存储位置；这一位的位权由存储约定决定

加减乘从低位向高位处理方便，所以常用低位在前。输入的字符 `'7'` 先减 `'0'` 才成为数值 7；输出时需按正确方向读回。

每一位的三个角色

角色	含义	变化
a[i]、b[i]	当前位数字，不存在的位当0	随下标向高位走
carry	上一位带来的进位	本位总和除10取整数部分
本位结果	只允许0…9	本位总和对10取余

算一位的方法
t=本位a+本位b+旧carry；本位存 t%10，向下一位带 t/10。循环到两个数的所有位都处理完；最后仍有进位就再添一位。

“高精度”在这些题里通常指大整数按位模拟，数组并不天然有无限长度。要留出最高位进位空间，结果为 0 时也至少保留一位 0。

两数位数 m、n，加法时间 O(max(m,n))，结果最多 max(m,n)+1 位。

独立任务 T25
把 798 与 27 按低位在前保存，逐位记录“本位总和、写入数位、新进位”，最后还原答案。再说明 999+1 会多出哪一格。

回到原卷：2509 选15；2512 选15；考纲第22页高精度。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

26.1 从1000减去1：借位怎样一格一格传过去?

先规定A≥B，使用一个变量borrow记录“低一位向本位借走了1”。借位不凭记忆直接跳过一串0，而是在每一位都执行相同的动作。

本位先算t=a[i]−b[i]−旧borrow。若t<0，就向更高一位借1，当前差加10，新borrow设1；否则不再借，新borrow设0。本位结果保存调整后的差。

1000−1 的本位	先扣旧借位	是否继续向高位借	保存的数字 / 新borrow
个位：0−1	0−1−0=−1	需要，加10	9 / 1
十位：0−0	0−0−1=−1	需要，加10	9 / 1
百位：0−0	0−0−1=−1	需要，加10	9 / 1
千位：1−0	1−0−1=0	不需要	0 / 0

暂存结果[9,9,9,0]表示0999。最高的多余0去掉后是[9,9,9]，输出999。三个0不是被一次神秘操作变成9，而是连续三轮都发现“本位还欠一个”。

为什么减数读完了，还不能结束?

减数1在个位就读完了，但borrow仍是1。如果直接复制A剩下的十、百、千位，就把这笔借位债忘掉。要处理完A的所有位，缺失的减数位按0参与。

> **一个借位变量已经够，不要又重复减数组**
> 采用borrow法时，借位通过下一轮减borrow体现；若同时把a[i+1]减1，下一轮又扣borrow，就扣了两次。直接改数组和记录borrow是两种表示，不能不加区分地叠用。

结果为0时，也要保留一个0；不能把所有零删到数组为空。完成后用“差+减数=被减数”核验，借位过程中还应检查最终borrow是否回到0。

> **接着往下读**
> 下一页解释A<B的符号与去零条件。T26用2000−7验证你能独立走完借位链。

26.2　高精度减法：借位也需要穿过 0

这一节要弄懂什么
一次借位影响相邻高位；跨越多个 0 时，后续各位必须继续处理，不能直接抄剩余位。

先比较大小，再处理符号

本页先讨论 A≥B 的非负结果。若 A<B，要另行处理符号或交换计算，不能指望无符号按位减法自动生成负数。

两种借位表示，只选一种理解

方法	当前位不足时	下一位怎样知道
借位变量 borrow	差加10，borrow设1	下一位减去borrow
直接修改被减数	当前位加10，高一位减1	下一轮看到已经被减过的数位

关键例子：1000−1
个位 0 不够，向十位借；十位原来是0，被借后也不够，需继续向百位借。处理必须走过所有被减数位，不能因较短的减数已经结束就停止借位传播。

删多余高位零，至少保留一格

低位在前时，多余最高位零在数组尾部。条件应包含“结果位数大于 1”。100−100 的结果是 [0]，不是空数组。

位数不同先比较位数，位数相同从最高位逐位比较。非负整数通常规范为没有多余前导零的形式。

核验：计算出的差 + 减数 = 原被减数

独立任务 T26
独立手推 2000−7 的每一位借位，记录每轮 borrow。再给出“借位链”“结果为零”“最小一位数”三类自测。

回到原卷：2406 选12；2412 选15；2606 选15。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

27.1 先展开乘法，再让数组替你收集同位的部分积

先用熟悉的算式看23×14。23是20+3，14是10+4，展开后有20×10、20×4、3×10、3×4四项。数组乘法只是按位权，把这些部分积放进对应格子。

拆开的乘积	等价的“多少个某单位”	低位在前放入
3×4=12	12个一	c[0]累加12
20×4=80	8个十	c[1]累加8
3×10=30	3个十	c[1]再累加3
20×10=200	2个百	c[2]累加2

两个不同乘积都属于十位，因此暂存格c[1]要“累加”，不能第二次赋值时把第一次的8盖掉。全部累加后得到[12,11,2]。

暂存格还不是最后的数位

12个一，留下2个一，进1个十；十位原有11，收到这1后变成12，于是留下2个十，再进1个百。百位由2变3，最后是[2,2,3]，读回322。

累计：[12,11,2]

处理个位：[2,12,2]

处理十位：[2,2,3] → 322

为什么格子编号相加?

个位编号0、十位编号1、百位编号2。十位×十位是百位，对应1+1=2；百位×十位是千位，对应2+1=3。一般地，$10^i \times 10^j = 10^{i+j}$，所以落在i+j。

若a有m位、b有n位，每一位a都要和每一位b配对，共m×n个数位乘积。两层循环在这里确实对应这两组独立选择，因此朴素乘法时间为O(mn)。

这一页先采用“全部累计，再统一进位”的版本。有的代码边乘边进位，读题时要认出它采用哪种约定，不能把两种版本的进位位置混接。

接着往下读

下一页整理位权、空间和复杂度；T27换成32×26，由你画出每项去向。

27.2　高精度乘法：位权决定放进哪一格

这一节要弄懂什么
不是背 i+j，而是先看个位、十位、百位的乘积落在哪里。

为什么贡献到 c[i+j]?

低位在前时，a[i] 的位权是 10^i，b[j] 的位权是 10^j；相乘的位权为 10^{i+j}，所以贡献累加到 c[i+j]。很多乘积会落在同一格，需要相加。

23×14 的部分积	位权	放在哪里
3×4=12	个位×个位	c[0] 加12
2×4=8	十位×个位	c[1] 加8
3×1=3	个位×十位	c[1] 加3
2×1=2	十位×十位	c[2] 加2

先累计，再统一进位
暂存格可以大于9，但最终每格必须为0…9。例中原始累计为 [12,11,2]：个位留2进1，十位变12再留2进1，百位变3，得到322。

m 位乘 n 位最多 m+n 位。朴素双重循环要做 m×n 个数位乘积，因此时间 O(mn)，不是只由较长的那一个长度决定。

暂存格的最大累计值也要估算。位数很大时，单格累加可能溢出；选择足够宽的类型或及时处理进位。最后规范最高位的 0。

独立任务 T27
按上面的“部分积→暂存格→进位”三步计算 32×26，注明每项为什么落入这一格；最后用普通乘法核验。

回到原卷：2409 选14；2503 选15；考纲高精度乘法。先独立做，再说依据。

☐ 能解释变量角色　☐ 能手推或举反例　☐ 隔天换题仍会

28.1 余下的3个百，落下一位后怎样继续分？

算329÷6，像分物品一样从最高位开始。先看3个百，不够分成每份1个百，所以商的百位是0，余下3个百。

落下十位的2时，不是把3和2相加。3个百相当于30个十，再加2个十，共32个十；所以本轮待分数是3×10+2=32。

落下哪一位	本轮待分数	商的这一位	新余数
百位3	0×10+3=3	0	3
十位2	3×10+2=32	5	2
个位9	2×10+9=29	4	5

余下的2个十，到了个位就是20个一，再加9得到29；可分出每份4个一，剩5。商位顺序是0、5、4，去掉最前面的多余0，得到54，余5。

rem到底属于哪一轮？

一轮开始时rem是之前已处理前缀的余数。乘10并落下一位后，变成这轮待分数；除完后更新rem，才成为交给下一轮的新余数。别把它与已写下的商混在一起。

```
current = rem * 10 + digit;
quotient_digit = current / divisor;
rem = current % divisor;
```

上一轮0≤rem<divisor，digit在0…9，所以current<10×divisor。本轮商一定在0…9，刚好是一位十进制数字。这也说明每轮应把余数真正更新好。

两个检查一起用

数值检查：329=6×54+5。范围检查：0≤5<6。只满足第一条还不够，例如把商少写1、余数多加6，等式仍成立，但余数已经没有分尽。

接着往下读

下一页沿用同样的竖式，解释除数也装不进普通整数时，怎样确定一位商。

28.2 除数也很长时，每一位商仍只有0到9

当除数本身也要用数组保存，仍然从高位向低位落数。区别是：不能直接用普通整数除法算current/divisor，需要用“大数比较和减法”来确定这一位商。

用1234÷12演示流程；这两个数本来装得下普通整数，选它们只是为了能在纸上看清同一套动作。

落下数位后	本轮待分数	试出一位商	减去除数×商位后
落下1	1	0，因为1<12	余1
落下2	12	1，因为12≤12<24	余0
落下3	3	0，因为3<12	余3
落下4	34	2，因为24≤34<36	余10

商位依次0、1、0、2，规范后是102，余10。中间的0占着十位位置，不能删掉；只有商最前面的多余0才不改变数值。

怎样试出这一位商?

一种朴素方法：从商位0开始，只要当前余数≥除数，就再减一次除数并把商位加1。每轮最多减9次，因为落下一位后的待分数严格小于除数的10倍。

比较时先看有效位数，位数相同再从最高位比较；减法就使用上一个专题的借位过程。这样高精度除法把已经学过的比较、减法和数位移动连了起来。

> **每轮先落一位，而且只落一次**
> 开始余数为0。每读到一个新数位，余数乘10再加这个数位，然后确定商位。若先把某个前缀装进余数，又从头重复落入这些数位，就把被除数重复拼接了。

被除数有m位、除数有n位时，上述朴素版本每一位最多做常数次O(n)的比较与减法，总体可按O(mn)估算。除数必须是正数；整套过程仍用A=B×Q+R且0≤R<B核验。

> **接着往下读**
> 下一页汇总大整数除小整数与除大整数的异同。T28检验你能否保留商中间的0。

28.3 高精度除法：从高位落下一位，余数接着走

这一节要弄懂什么
除法方向与加减乘的常用处理方向不同。先解释竖式中的余数，再理解代码。

大整数除以正的小整数

从最高位到最低位，设上一轮余数 rem，本轮读入数字 digit：本轮待分数为 `rem×10+digit`；商的这一位是它除以 b 的整数部分，新余数是它对 b 取余。

1234÷6	本轮待分数	商位	新余数
落下1	1	0	1
落下2	1×10+2=12	2	0
落下3	0×10+3=3	0	3
落下4	3×10+4=34	5	4

余数的检查线
每轮结束满足 0≤rem<b。最终 1234=6×205+4；中间的商位0不能丢，只有商最前面的多余0可以去掉。除数为0不合法。

若除数也是大整数

仍从高位到低位做长除法，余数也用数组存。落下一位相当于“余数乘10再加这一位”；通过大整数比较和减法确定本位商 0…9。依此朴素做法，m 位被除数与 n 位除数可按 O(mn) 估算。

大整数除小整数，按固定宽度小整数运算计，时间 O(m)。即使余数小于 b，`rem*10` 也要用足够宽的类型。

独立任务 T28
手推 1005÷5，记录每次落位后的商位和余数，保留商中间的0。再用“除数×商+余数”核验；用 3÷7 检查商为0的情况。

回到原卷：2506 选15（仅取竖式概念，见附录）；2603 选15。先独立做，再说依据。

☐ 能解释变量角色 ☐ 能手推或举反例 ☐ 隔天换题仍会

迁移验收 · 单选 1–8

自编迁移题，并非原卷摘录。每题选择一个答案，并至少为不确定的题写一句理由。可分两次做，不计时起步。

1. 已知单链表结点 p，需在 p 之后插入 s。为了不丢失原后继，先做哪件事?

A. 让 p 指向 s
B. 让 s 记住 p 的原后继
C. 释放 p
D. 令 s.next 指向 s

选择：____ 依据 / 待核对：_____

2. 删除 dummy→5→5→8 中全部 5，删掉第一个 5 后，cur 应怎样变化?

A. 前进到第二个5
B. 前进到8
C. 继续留在dummy
D. 设为空

选择：____ 依据 / 待核对：_____

3. 用 b==0 为终点的辗转相除法，gcd(42,30) 下一次调用是?

A. gcd(12,30)
B. gcd(30,12)
C. gcd(30,42)
D. gcd(42,12)

选择：____ 依据 / 待核对：_____

4. 下列说法正确的是?

A. 1是素数
B. 所有奇数都是素数
C. 2是唯一偶素数
D. 互质的两数必须各自是素数

选择：____ 依据 / 待核对：_____

5. 埃氏筛在处理素数5时，可从25开始标记，主要因为?

A. 小于25没有合数
B. 较小的5的合数倍数已被更小质因子标记
C. 5没有更大倍数
D. 保证所有合数只被标记一次

选择：____ 依据 / 待核对：_____

6. 线性筛处理 i=9，依次试 p=2、3。标记27后应停止的原因是?

A. 9==3
B. 9%3==0
C. 27不是合数
D. 2和3相邻

选择：____ 依据 / 待核对：_____

7. 查 [1,4,4,7] 第一个 ≥5 的位置，使用0起下标，应返回?

A. 1
B. 2
C. 3
D. 4

选择：____ 依据 / 待核对：_____

8. 最多切K刀，使所有段长≤x。x增大时，可行性方向是?

A. 可行会变不可行
B. 不可行可能变可行
C. 必须交替变化
D. 与x无关

选择：____ 依据 / 待核对：_____

迁移验收 · 单选 9—15

自编迁移题，并非原卷摘录。每题选择一个答案，并至少为不确定的题写一句理由。可分两次做，不计时起步。

9. 函数每次只递归调用一次 f(n/2)，其余 O(1)，n≤1返回。时间与栈空间分别是?

A. O(n)、O(n)　　B. O(log n)、O(log n)

C. O(n log n)、O(n)　　D. O(1)、O(1)

选择：____　依据 / 待核对：_____

10. 朴素快速幂把 power(x,n/2) 写了两次，且不缓存。相比只算一次复用，下列正确的是?

A. 调用次数相同　　B. 只影响返回值

C. 重复子调用可能把时间从O(log n)增至O(n)　　D. 两者一定均O(1)

选择：____　依据 / 待核对：_____

11. 标准归并把7个元素分成单元素再合并，merge共调用几次?

A. 3　　B. 6

C. 7　　D. 14

选择：____　依据 / 待核对：_____

12. 升序数组用总选首元素的普通快排，最坏时间为?

A. O(1)　　B. O(log n)

C. O(n log n)　　D. $O(n^2)$

选择：____　依据 / 待核对：_____

13. 面额1、5、7凑10，最大面额优先得到几枚？能否据此保证最少?

A. 2枚，能　　B. 4枚，不能

C. 3枚，能　　D. 10枚，不能

选择：____　依据 / 待核对：_____

14. 高精度加法某位数字8、7，上一位进位1。本位数字和新进位为?

A. 6与1　　B. 1与6

C. 16与0　　D. 6与0

选择：____　依据 / 待核对：_____

15. 大整数除以7，旧余数3，落下数字5。本位商与新余数为?

A. 3与5　　B. 5与0

C. 0与5　　D. 35与0

选择：____　依据 / 待核对：_____

迁移验收 · 判断 1—10

每题先判断，再给一个理由；认为错误时尽量举反例，而不只改一个字。

1. 链表插入只要改几个指针，所以按值查找插入位置再插入，整个过程一定是 O(1)。

☐ 对　☐ 错　依据：_____

2. 遍历非空、无哨兵的循环链表，可以在走过一轮后重新回到 head 时停止。

☐ 对　☐ 错　依据：_____

3. 两个正整数同余于模 m，等价于它们的差能被正整数 m 整除。

☐ 对　☐ 错　依据：_____

4. 唯一分解定理要求分解得到的每个质因数都不同。

☐ 对　☐ 错　依据：_____

5. 有终止条件的递归一定会结束，不需要检查参数怎样变化。

☐ 对　☐ 错　依据：_____

6. 标准归并排序的 merge 仅调用 n−1 次，所以整个排序时间是 O(n)。

☐ 对　☐ 错　依据：_____

7. 普通快速排序改用随机基准后，就变成稳定排序。

☐ 对　☐ 错　依据：_____

8. 只知道问题有最优子结构，就足以证明任意局部最优选择都是正确的。

☐ 对　☐ 错　依据：_____

9. 高精度减法处理完短的减数后，可以不再处理高位借位。

☐ 对　☐ 错　依据：_____

10. 去掉商前面的多余0时，若结果为0，也应保留一个0。

☐ 对　☐ 错　依据：_____

复盘卡：把一道题变成一个检查动作

在订正后填写。先保留孩子自己的表达；还没订正的题只记“待复盘”，不替孩子写原因。

基本信息	孩子填写
题源 / 题号	_____
状态	☐ 待复盘 ☐ 已订正 ☐ 隔天复测通过
原来判断 / 修改后判断	_____

我错在哪里?

我为什么会这样想?

下次遇到类似题，我会先检查什么?

一个能区分新旧想法的反例

> **检查动作可以很具体**
> 例如“先画旧后继再改箭头”“相等时问中点能否保留”“分别数调用总数和最长路径”。这里只给动作类型，真正的反思由孩子自己写。

复测日期：_____ 换题编号：_____ ☐ 能讲出依据 ☐ 仍需提示

九卷小题索引 · 2406 / 2409

“选”=单选，“判”=判断；每题归入一个主专题，复杂度等交叉能力仍在各专题讲解。题号只作定位，不在孩子页展示原卷答案。

2406 · 2024年6月 · 25题

主专题	单选题号	判断题号
初等数论	5、7、8	4、5
高精度运算	12	—
链表	3、4	1、2、3、10
二分算法	11	—
递归与优化	1、6、14、15	—
分治与排序	9、10、13	7、8、9
贪心算法	2	6

2409 · 2024年9月 · 25题

主专题	单选题号	判断题号
初等数论	4、5、6、7	2、3
高精度运算	14	—
链表	1、2	1
二分算法	13	8
递归与优化	3、8、15	9、10
分治与排序	9、10、12	5、6、7
贪心算法	11	4

九卷小题索引 · 2412 / 2503

“选”=单选，“判”=判断；每题归入一个主专题，复杂度等交叉能力仍在各专题讲解。题号只作定位，不在孩子页展示原卷答案。

2412 · 2024年12月 · 25题

主专题	单选题号	判断题号
初等数论	5、6、7、8	2、3
高精度运算	15	—
链表	1、2、3	1
二分算法	11、12	8、9
递归与优化	4	5、10
分治与排序	9、10、14	6、7
贪心算法	13	4

2503 · 2025年3月 · 25题

主专题	单选题号	判断题号
初等数论	4、5、6	3
高精度运算	15	—
链表	1、2、3	1、2
二分算法	11、12	8
递归与优化	7、8	5
分治与排序	9、10、14	6、7、10
贪心算法	13	4、9

九卷小题索引 · 2506 / 2509

“选”=单选，“判”=判断；每题归入一个主专题，复杂度等交叉能力仍在各专题讲解。题号只作定位，不在孩子页展示原卷答案。

2506 · 2025年6月 · 25题

主专题	单选题号	判断题号
初等数论	5、6、7、8	1、2、3、10
复杂度综合	10	—
高精度运算	15	—
链表	1、2、3、4	—
二分算法	11、12	6
递归与优化	—	9
分治与排序	9、14	4、5、8
贪心算法	13	7

2509 · 2025年9月 · 25题

主专题	单选题号	判断题号
初等数论	4、5、6、7、8、9	1、2、6
高精度运算	15	—
链表	1、2、3	4
二分算法	10	5
递归与优化	—	3、8、9
分治与排序	11、12、14	7
贪心算法	13	10

九卷小题索引 · 2512 / 2603

“选”=单选，“判”=判断；每题归入一个主专题，复杂度等交叉能力仍在各专题讲解。题号只作定位，不在孩子页展示原卷答案。

2512 · 2025年12月 · 25题

主专题	单选题号	判断题号
初等数论	4、6、7	2、4
高精度运算	15	—
链表	1、2、3	1、3
二分算法	11、12	5
递归与优化	5、13	8、9
分治与排序	8、9、10	6
贪心算法	14	7、10

2603 · 2026年3月 · 25题

主专题	单选题号	判断题号
初等数论	4、5、6	6、9
复杂度综合	—	4、7
高精度运算	15	—
链表	1、2、3	1
二分算法	7、8、10	2
递归与优化	9	10
分治与排序	11、12、13、14	3、5
贪心算法	—	8

九卷小题索引 · 2606

“选”=单选，“判”=判断；每题归入一个主专题，复杂度等交叉能力仍在各专题讲解。题号只作定位，不在孩子页展示原卷答案。

2606 · 2026年6月 · 25题

主专题	单选题号	判断题号
初等数论	4、5、6、8	3、9
复杂度综合	—	8
高精度运算	15	—
链表	1、2、3	1、2
二分算法	9、10	7
递归与优化	7	—
分治与排序	11、12、13	4、5、10
贪心算法	14	6

九卷主专题题量（合计225）

专题	题量	阅读说明
初等数论	54	主专题归类；非未来命题预测
复杂度综合	4	主专题归类；非未来命题预测
高精度运算	9	主专题归类；非未来命题预测
链表	40	主专题归类；非未来命题预测
二分算法	25	主专题归类；非未来命题预测
递归与优化	26	主专题归类；非未来命题预测
分治与排序	48	主专题归类；非未来命题预测
贪心算法	19	主专题归类；非未来命题预测

索引使用方法

学完一个主题，只选对应 1–2 题验证。先做未见过的题更能检验迁移；遇到与讲义规则冲突的原题，先查家长附录的材料校勘。

家长附录 · 使用边界与核对方式

以下为家长备课参考。网页默认折叠；PDF 从此页起建议单独保管。

先观察证据，再判断掌握

孩子四级已通过，但五级每一专题的掌握情况应由新作答验证。不能把四级历史错点或尚未学习的知识，直接写成她五级的能力缺陷。

每次围绕一个核心任务展开。答对后，选“解释依据”或“构造反例”核实，不连续拆成机械填空。明显疲劳时保留一张思路卡，第二天从模型重新开始。

三类能力一起看

表现	诊断方向	当下问题示例
会名词，不会手推	模型还没落到变量	“这轮 cur 代表谁？”
会手推，边界出错	停止条件或区间含义不清	“只剩一个候选，还要不要检查？”
原例会，换题不会	记住表面格式，迁移不足	“这题和上一题不变的动作是什么？”
做对却说不出理由	暂记为待验证，不记为掌握	“给一个能推翻另一个选项的例子。”

原卷有参考答案，不等于代码处处严谨

本讲义不复刻九卷答案表。按真实模型解释概念，对明显瑕疵标明题号与原因；不让孩子为解释错误代码而反复猜题人意图。需要正式计分时，应按考试方确认的标准，不能把本稿当官方勘误。

材料范围与可追溯性

知识依据为用户提供《考纲.pdf》第22–24页；题型覆盖为九份本地 PDF 的每卷15道单选、10道判断。未引用编程大题作为小题统计。全部225个题号在前面的索引中各出现一次。

讲义中的 T01–T28、迁移验收题均为教练自编或重设数字的教学例，不冒充真题原文。表中“通常实现”“正整数”“固定宽度运算”等限定应与结论一起使用。

家长核对 · T01—T07

仅在孩子已经尝试后核对。下列是证据标准，不是要孩子逐字复述的标准答案。

T01 · 复杂度：数工作量，不数代码行数

应看到的证据：i 为 1、2、4、8、16，共 5 次；嵌套内层 n 次，总体 O(n log n)，不是 O(log n)。

卡住时：先让她写出实际 i 值，检查她数的是轮数还是内层总工作。

T02 · 递归：把同一种小任务交给下一层

应看到的证据：F(n)=n!；F(0)=F(1)=1；F(n)=n×F(n−1)。4→3→2→1，回传 1、2、6、24。

卡住时：若只会写乘法，追问 F(3) 替 F(4) 完成了哪一部分。

T03 · 递归追踪：输出、返回值、调用次数是三件事

应看到的证据：输出 3 2 1 1 2 3；共 4 次调用；含基例时最大同时 4 层。时间、栈空间均 O(n)。

卡住时：让她在“调用下一层”处画等待标记，返回后从下一句继续。

T04 · 递归优化：相同的事别重新算

应看到的证据：重复版 1+2+4+8=15 次，复用版 4 次；两者最长路径均 8→4→2→1，为 O(log n)。

卡住时：先问她两次相同表达式究竟有没有一个变量替它保存结果。

T05 · 数论起点：约数、倍数、素数与奇偶

应看到的证据：1 两者皆非；2、53 为素数；9、49 为合数。53 只需排除 2、3、5、7。25 是反例。

卡住时：若她漏 49，重点是为什么试除上界包含等号，而不是多做几次除法。

T06 · GCD 与 LCM：铺方砖和再次相遇

应看到的证据：(12,18)→(18,12)→(12,6)→(6,0)，GCD=6；再次同时闪是 LCM=36 分钟。

卡住时：若一开始 a<b，她应能发现首次取余为 a，下一次自然交换到大数在前。

T07 · 质因数分解：种类、次数与约数个数

应看到的证据：$90=2\times3^2\times5$；3 种；3 出现 2 次；约数个数 2×3×2=12。

卡住时：让她给三个“数量”分别写名字，避免正确算出了另一个问题。

家长核对 · T08—T14

仅在孩子已经尝试后核对。下列是证据标准，不是要孩子逐字复述的标准答案。

T08 · 同余与模运算：余数相同，差能整除

应看到的证据：差 24；大于 1 的正约数为 2、3、4、6、8、12、24，恰为所求模数。

卡住时：若她不断试模数，提示同余意味着差可以分成完整的若干圈。

T09 · 埃氏筛：用素数划掉它的倍数

应看到的证据：如 12、18、24 同时被 2 和 3 标记；6 已被 2 标记，9 是 3 尚需处理的最早倍数。

卡住时：不要只接受“因为效率高”，需要她说出前面的倍数为什么已被处理。

T10 · 线性筛：让每个合数只登记一次

应看到的证据：p=2 标记 12，6%2==0 后停止；若继续 p=3 会标记 18，但它应由 i=9、p=2 生成。

卡住时：要求她把“i 的因子”和“i×p 的最小质因子”分开说。

T11 · 链表：数据盒子之间用箭头相连

应看到的证据：p 从 7 结点移动到 4 结点，随后该结点数据改成 6；head 和 next 链接均未变。

卡住时：若她把箭头也改了，回到“p 变量”和“p 指向的结点的 next 字段”两个盒子。

T12 · 链表插入：接好新结点，再改旧入口

应看到的证据：先保留 B 的原前驱 A；S.next=B，S.prev=A，再 A.next=S、B.prev=S。顺序可有多种，只要使用旧指针前不覆盖它。

卡住时：不用要求背唯一顺序，让她在每一步之后重画箭头检验。

T13 · 链表删除：先绕过去，再释放

应看到的证据：cur 始终先留在 dummy，连续删除两个 5 后才越过 8；最终 dummy.next 指向 8。否则可能漏掉第二个 5。

卡住时：聚焦 cur 是“前驱”这个角色，别把它当正在删除的人。

T14 · 循环、反转与快慢指针：先说明停止的原因

应看到的证据：第一轮后 prev=A、cur=B；第二轮 prev=B、cur=C；第三轮 prev=C、cur=空；新链 C→B→A。必须先存旧 next。

卡住时：这里讲反转服务于箭头角色理解，不要求立刻写完整模板。

家长核对 · T15—T21

仅在孩子已经尝试后核对。下列是证据标准，不是要孩子逐字复述的标准答案。

T15 · 二分查找：每次排除不可能的一半

应看到的证据：找13：10、16、13；找12：10、16、13 后空区间。n=1 时 l<r 会直接漏查唯一元素。

卡住时：别让她背 mid±1；问中点是否已经被证明确实不可能。

T16 · 二分边界：中点还可能是答案，就保留它

应看到的证据：位置分别为 1、4、5；5 为 n 标记。上中位确保保留 mid 到左边界时仍能收缩。

卡住时：能答下标不算通过，要说明相等时为什么还向左走。

T17 · 二分答案：猜一个要求，检查做不做得到

应看到的证据：x=3 需3刀，不可行；x=4、5 各需2刀，可行。最小值4。2刀最多3段，不等于2段。

卡住时：先让她画实际分段；原卷2512文字有刀与段混用，见附录。

T18 · 分治：分开求，最后必须接得起来

应看到的证据：左最佳4，右最佳3，跨界4+3=7。全负非空段答案−2，初始0可能错误允许了空段。

卡住时：只提醒“有没有跨中点的连续段”，让她自行发现漏掉的类别。

T19 · 归并排序：先让两队有序，再比较队首

应看到的证据：应为2A、2B、4、5；相等先取右会变成2B在2A前，破坏稳定性。

卡住时：先让她说稳定性检查的是数值还是相等元素的原始身份。

T20 · 快速排序：一轮划分，不等于排完

应看到的证据：交换4和2得[3,1,2,4]，相遇在下标2，基准归位得[2,1,3,4]；左边2、1仍无序。

卡住时：要求她标出原基准的存储位置，以及临时保存的基准值。

T21 · 快排为何退化：划分是否平衡很重要

应看到的证据：长度依次5、4、3、2、1；每轮仍要扫描，累计呈平方增长。输入有序并不自动使该快排提前停止。

卡住时：让她数真实剩余区间长度，不要套用“二分所以对数”。

家长核对 · T22—T28

仅在孩子已经尝试后核对。下列是证据标准，不是要孩子逐字复述的标准答案。

T22 · 排序核对表：次数、时间、稳定性分别算

应看到的证据：合并4次；例如[1,3,5]与[2,4,6]，比较5次后剩余6直接追加。

卡住时：若她把n−1当总运行时间，就挑“合并两个长队”检查单次成本。

T23 · 贪心：当前最优，需要一个“不吃亏”的理由

应看到的证据：贪心6+1+1共3枚，4+4共2枚；该面额系统下贪心不能保证最少。

卡住时：孩子只需给出足以推翻的反例，不要求当场学习完整替代算法。

T24 · 四类贪心：每次保留后面的机会

应看到的证据：[2,4]、[3,3]，共2盒；成功时左右指针都动。剩一个时单独计1盒。4若连最轻也配不上便必须独装。

卡住时：先问每一轮究竟消耗了哪几件，再让她决定哪些指针变化。

T25 · 高精度与加法：数组保存每一位

应看到的证据：个位8+7=15，写5进1；十位9+2+1=12，写2进1；百位7+0+1=8，写8；结果825。999+1新添千位1。

卡住时：若孩子把15直接塞进一格，回到“一格只保存一个十进制数字”的约定。

T26 · 高精度减法：借位也需要穿过 0

应看到的证据：结果1993；从个位起borrow依次1、1、1、0。可选1000−1、5−5、8−3作为三类自测。

卡住时：原2406选12的完整代码有借位传播问题，任务使用本页完整过程，不照抄原代码。

T27 · 高精度乘法：位权决定放进哪一格

应看到的证据：低位数组[2,3]与[6,2]，累计[12,22,6]，进位后[2,3,8]，结果832。

卡住时：若她记成i×j，用十位乘十位落百位的位权关系检查。

T28 · 高精度除法：从高位落下一位，余数接着走

应看到的证据：商位0、2、0、1，规范后201，余0；3÷7商0余3。两式均符合A=bq+r且0≤r<b。

卡住时：让她解释为什么余数乘10，而不是把旧商位乘10。

家长核对 · 迁移验收单选

单选15题、判断10题，每题2分可计50分。但本套是配套学习验收，未经难度标定，不能用分数直接预测考试是否通过。未学完时按专题选题。

单选	答案	核对理由
1	B	先保存旧后继，再改入口；对应12课。
2	C	cur 是待删结点的前驱，继续检查新后继；13课。
3	B	下一对为(b,a%b)；6课。
4	C	定义与反例：9、8和9；5课。
5	B	10、15、20 已筛过；9课。
6	B	保持由最小质因子唯一筛除；10课。
7	C	值7位于下标3；16课。
8	B	x增大要求放宽；17课。
9	B	只有一条减半调用链；3—4课。
10	C	要数整棵调用树；4课。
11	B	7组变1组，每合并一次少1组；22课。
12	D	每次只减少一个元素，逐层扫描；21课。
13	B	7+1+1+1为4枚，而5+5为2枚；23课。
14	A	总和16，个位留6向高位进1；25课。
15	B	35÷7商5余0；28课。

先做再看

这里核对的是配套自编题；真题核对须回到原卷，并结合材料校勘。不要把一个得分直接写成永久掌握状态。

家长核对 · 迁移验收判断

答“错”时，应指出具体不成立的部分，最好给一个反例。答“对”时，要能说出前提。

判断	结论	核对理由
1	错	定位可能需O(n)。
2	对	须先处理并前进，区别于开始就判断p!=head。
3	对	同余定义；这里m>0。
4	错	允许重复，例如12=2×2×3。
5	错	参数还必须走得到终点。
6	错	单次merge不是恒定工作量。
7	错	基准策略不保证相等元素顺序。
8	错	还需贪心选择性质的论证。
9	错	1000−1需要跨0连续借位。
10	对	零需要合法表示。

订正后再验证

先让孩子说出原来的依据，再指出关键偏差；让她重做一题同模型、不同表面的例子。反思页留给孩子填，不把这份答案复制成 kidReflection。

家长附录 · 原卷校勘 1

以下是对所提供材料的教学校勘，并非考试方发布的官方勘误。仅供家长备课；不把这些材料瑕疵记成孩子错误。

2406 选4；2412 选3

原材料问题：函数形参 head 按值传入，函数内重设 head 不会更新调用者的头指针。

讲义处理：横线考查局部链接，但完整操作若要改变外部头指针，应返回新头或采用恰当引用传参。

2406 选12

原材料问题：短减数结束后直接复制剩余被减数位，不能完成穿过0的借位。

讲义处理：用1000−1即可暴露问题；教学改用逐位处理全部被减数位的版本。

2406 选15

原材料问题：表达式 fun(n−2)−fun(n−1) 未由减号规定两个调用的先后求值顺序。

讲义处理：原答案按先左后右给输出；不能把这个顺序教成C++通则。练习使用分开的调用语句明确次序。

2406 判10

原材料问题：delete ptr 释放动态对象，不是删除指针变量本身；之后 ptr 是悬空指针。

讲义处理：不把“能打印地址”当作释放后使用指针的通用安全保证，更不能再解引用。

2409 选5；2412 选6；2503 选8

原材料问题：依次存在 void 函数返回 vector、有返回类型的函数空 return、循环阶乘漏 return res 等代码问题。

讲义处理：只讨论题目意图时注明；不复制为可运行模板。原卷中的变长局部数组等也不当作标准C++范例。

2409 选7

原材料问题：参考选项将试除分解的量级写成 O(√n log n)。

讲义处理：这是宽松上界。按常见固定宽度运算分析，试除候选共 O(√n)，成功除因子的总次数 O(log n)，整体可给更紧的 O(√n) 上界。

家长附录 · 原卷校勘 2

以下是对所提供材料的教学校勘，并非考试方发布的官方勘误。仅供家长备课；不把这些材料瑕疵记成孩子错误。

2409 选11

原材料问题：循环条件 i<j，结束后没有处理 i==j 的剩余一人。

讲义处理：可用于辨认贪心思想，不可当完整最少船数模板；用 n=1 就能检查漏计。

2503 选3

原材料问题：称“双向循环链表”，但给出的选项只完成 head.next=tail、tail.prev=head。

讲义处理：还不足以建立完整双向环；外侧链接也须明确。不要把它当作完整循环表初始化。

2506 选4

原材料问题：prev 初值为空，k=1 时内循环不执行，删除时会解引用空指针。

讲义处理：必须补充 k 的限制或正确初始化前驱；题目局部填空正确不表示全输入安全。

2506 选13

原材料问题：对 const vector 排序，且输出使用未定义的 sorted_coins。

讲义处理：可辨认大面额优先的贪心思想；完整代码有编译问题，且任意面额不保证最优。

2506 选14

原材料问题：A 把 i 说成“大于基准”的边界，实际上是 ≤pivot 区的末尾；D 又说快排稳定。

讲义处理：至少A与D都不正确，单选题不宜作为唯一答案验收；随机基准也不保证绝不出现最坏情形。

2506 选15

原材料问题：余数预装被除数前 b.len 位后，首次循环又乘10并落下已包含的数位。

讲义处理：如12÷3会先把1重复拼成11；完整除法流程有问题。只借此讨论“余数乘10、落下一位”，用本讲义正确竖式练习。

家长附录 · 原卷校勘 3

以下是对所提供材料的教学校勘，并非考试方发布的官方勘误。仅供家长备课；不把这些材料瑕疵记成孩子错误。

2509 选6

原材料问题：选项2*i和i*i均可作为正确筛除起点（数值不溢出时）。

讲义处理：i*i更省重复，给定答案取它；不能说2*i无法得到素数表。原题没有明确“最佳”，存在选项歧义。

2509 判2；2512 选12

原材料问题：前者 void 函数返回数值；后者题干说K段，注释与代码却按最多K刀。

讲义处理：LCM数值与返回类型分开看；切割教学统一按“最多K刀”，不要把刀数当段数。

2503 判1；2512 判3

原材料问题：先覆盖 next，再 delete p->next 会失去原待删后继，可能删除错结点。

讲义处理：如果讨论复制后继技巧，应先保存原后继的指针，再改链接，最后释放保存的结点。

2603 判5

原材料问题：只给跨区间扫描，没保证两边先有序，也没递归统计各半内部逆序对，更没合并排序。

讲义处理：不可宣称该片段独立统计任意区间全部逆序对；如[2,1 | 4,3]会漏掉两侧内部逆序。

各卷排序、筛法、复杂度判断

原材料问题：“稳定”“更快”“原地”“递归更省空间”等结论常被省略前提。

讲义处理：以具体实现和计量口径为准；教材量级模型也默认不发生整数溢出。对不严谨小题可讨论条件，不强迫孩子套口诀。